Chapter 1

Cellular Automata for In-memory Computing

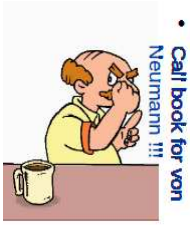
Hungarian Mathematician von Neumann -Invented modern computer architecture in 1946-48

Figure 1.1: John von Neumann

His computing unit relied on central processing unit (CPU) - CPU is now almost exhausted

Neumann also introduced Cellular Automata (CA) in 1950's - Each lattice point of CA is a computing unit

What was in Neumanns mind



CA is endowed with modeling natural/physical systems

Effective applications of CA can be found in

- Modeling of Physical Systems, Pattern Recognition

Classification, Data Compression

Cryptosystem, Authentication

Error Correcting Codes, Circuit Testing

Sensor Network, Cellular Mobile Network

It leads to scalable HW architectures with very high device densities.

CA constitute inherently parallel computing paradigm of high efciency and robustness.

CA-based models are of regular structure with exclusively local interconnections.

It is justiiable to search for computing paradigms that combine capabilities and structural simplicity of CA with memristors

which show promise to be used for in-memory computing

This focuses on CA-based algorithm implementations using circuits comprising

memristors, composite memristive devices, and conventional electronic components.

CA originally postulated in 1940's by Ulam and von Neumann.

For designing computational models of physical systems where
space and time are discrete and interactions are only local.

A CA consists of a regular and uniform d-dimensional lattice (or array).

Each cell is normally restricted to local neighborhood interaction only.

Therefore, it is incapable of immediate global communication.

Neighborhood of cell is generally taken to be cell itself and some/all of immediately adjacent cells.

States at each cell are updated simultaneously at discrete time steps
based on states in their neighborhood at preceding time step.

Algorithm which is used to compute next cell state is referred to as CA local evolution rule.

New state of a particular cell i at time t+1 is only a function of previous state of the speciic cell and of the cells which belong to its designated neighborhood.

Neighborhood size n is usually taken to be n = 2r + 1, where r is a positive integer - known as radius -that is, number of cells on each side that are involved.

For two-dimensional (2-d) CA, two types of neighborhood are usually considered.

(i) von Neumann neighborhood - consists of cell i and its 4 geographical neighbors $\{$north, west, south and east$\}$

(ii) Moore (9) neighborhood - consists of cell i and its 4 geographical neighbors $\{$north, west, south, east, northeast, northwest, southeast and southwest$\}$

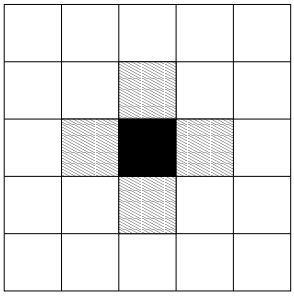


Figure 1.2: Two dimensional cellular automata (von Neumann neighborhood)

Moore neighborhood

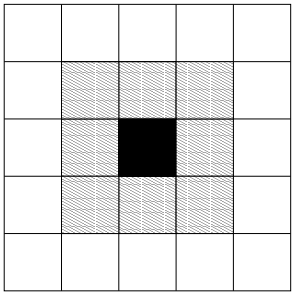


Figure 1.3: Two dimensional cellular automata (Moore neighborhood)

One-dimensional 3-neighborhood

Null boundary

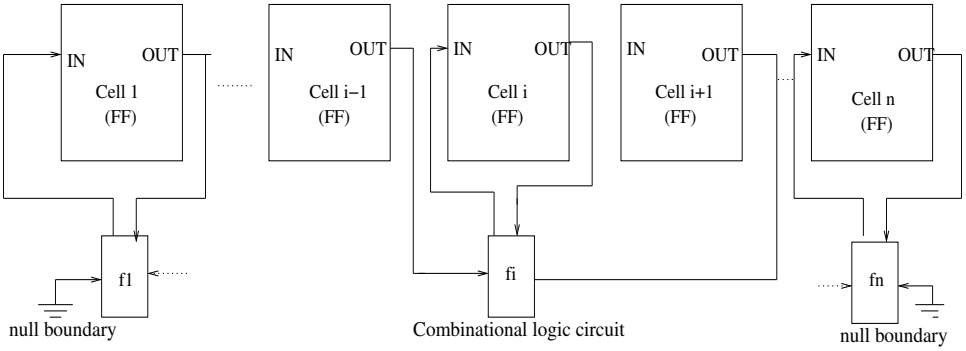


Figure 1.4: One dimensional null boundary cellular automata

Periodic boundary

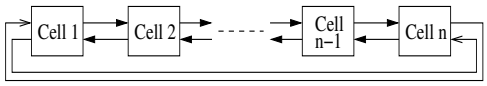


Figure 1.5: One dqit+1 = f (qit 1; qit ; qit+1)imensional periodic boundary cellular automata

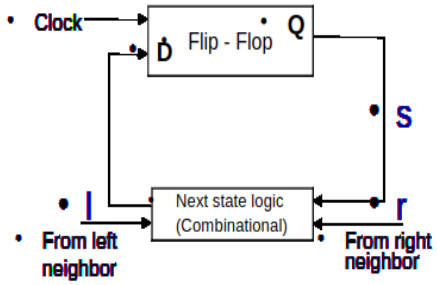


Figure 1.6: CA cell

$q_i^{t+1} = f_i(q_{i-1}^t, q_i^t, q_{i+1}^t$

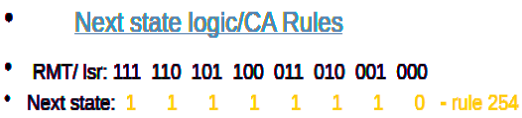


Figure 1.7: CA cell rule

Table 1.1: Additive CA Rules

With XOR (Linear CA)			With $XNOR$ (Complemented rule)		
rule 60	:	$q_i(t+1) = q_{i-1} \oplus q_i$	rule 195	:	$q_i(t+1) = \overline{q_{i-1} \oplus q_i}$
rule 90	:	$q_i(t+1) = q_{i-1} \oplus q_{i+1}$	rule 165	:	$q_i(t+1) = \overline{q_{i-1} \oplus q_{i+1}}$
rule 102	:	$q_i(t+1) = q_i \oplus q_{i+1}$	rule 153	:	$q_i(t+1) = \overline{q_i \oplus q_{i+1}}$
rule 150	:	$q_i(t+1) = q_{i-1} \oplus q_i \oplus q_{i+1}$	rule 105	:	$q_i(t+1) = \overline{q_{i-1} \oplus q_i \oplus q_{i+1}}$
rule 170	:	$q_i(t+1) = q_{i+1}$	rule 85	:	$q_i(t+1) = \overline{q_{i+1}}$
rule 204	:	$q_i(t+1) = q_i$	rule 51	:	$q_i(t+1) = \overline{q_i}$
rule 240	:	$q_i(t+1) = q_{i-1}$	rule 15	:	$q_i(t+1) = \overline{q_{i-1}}$

If all states in the state transition graph of a CA lie in cycles, it is called a group CA/invertable; otherwise it is a non-group/non-invertable CA.

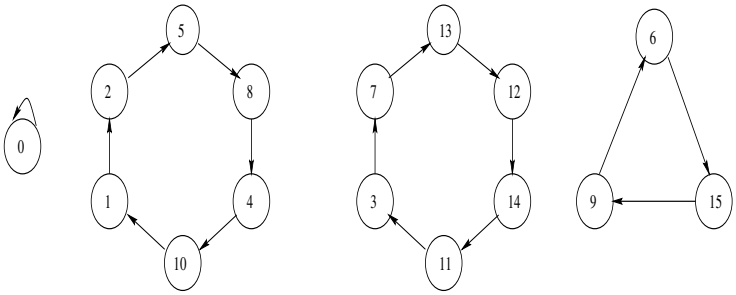


Figure 1.8: State transition diagram of a 4-cell non-maximal length group CA.

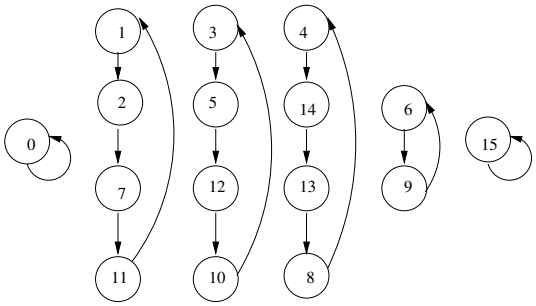


Figure 1.9: State transitions of a reversible $CA < 90, 150, 150, 90 >$

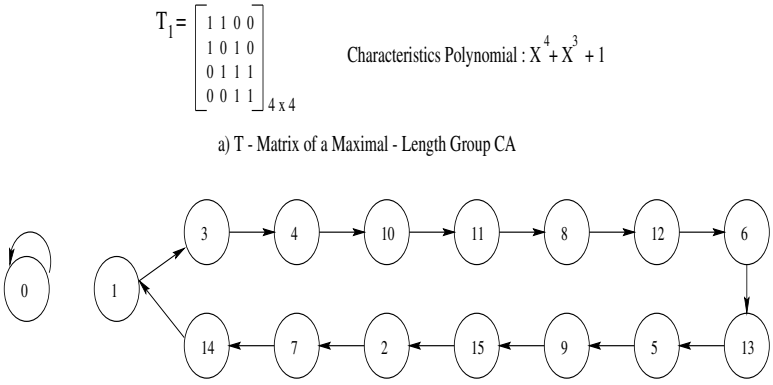


Figure 1.10: A 4-cell maximal length group CA.

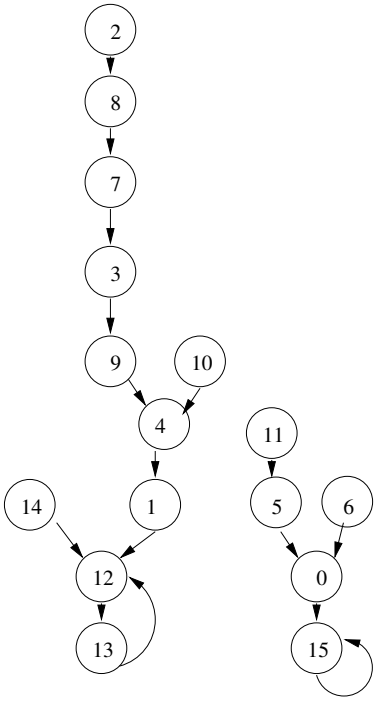


Figure 1.11: State transitions of an irreversible $CA < 105, 129, 171, 65 >$

In CA search space, a set of CA $states$ towards which neighboring $states$ asymptotically approach in course of dynamic evolution.

This set of $states$ forms *attractors* of CA $state$ space

An arratctor forms a $basin$ of attractions with the neighboring $states$

A *single length cycle attractor* is one where the number of $state$ of an $attractor$ is one

The attractors may be multi-length cycle where state count in the cycle is greater than one

Depth of a CA is the length of the longest path from any state to an attractor

MACA

No multi-length cycle only single length cycles

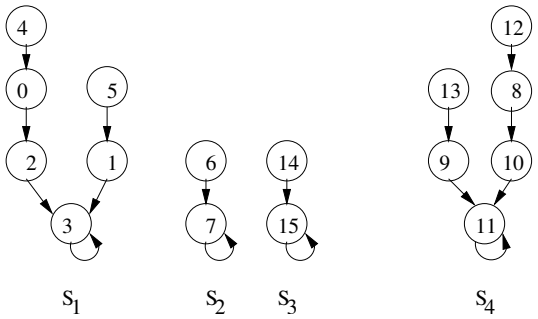


Figure 1.12: State transitions of a CA with rule vector $< 222, 221, 192, 68 >$

An *n-cell* MACA with k number of $attractors$ can be viewed as k-class natural classifier

SACA

A CA without a multi-length cycle and having one and only single length cycle attractor is SACA

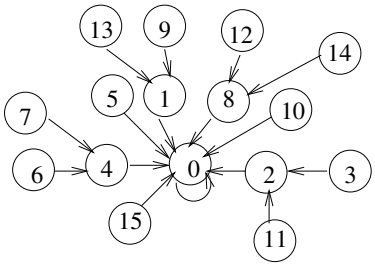


Figure 1.13: State transition diagram of SACA <64,64,64,64>

Two Attractor CA (TACA)

CA with exactly two single length cycle attractors and without a multi-length cycle is TACA

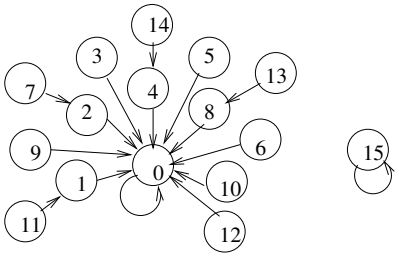


Figure 1.14: State transition diagram of TACA <128,128,128,128>

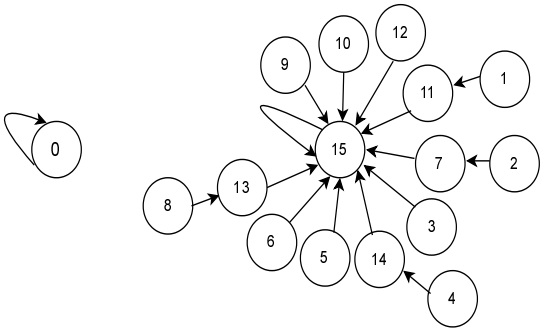


Figure 1.15: TACA with rule 254

TACA with rule 254 is employed for the In-Memory Search

Search

Key and data element XOR bit wise

Check output whether all 0s

For n-bit key n-bit output is to be checked

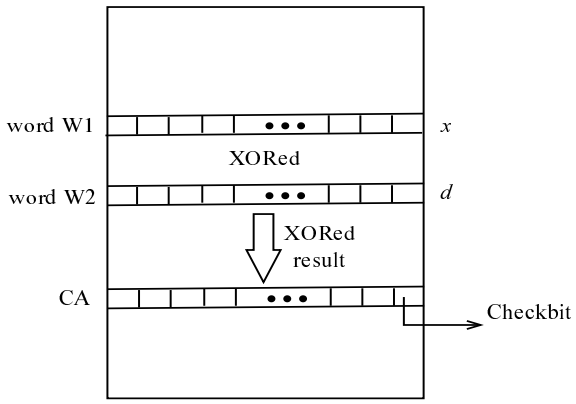


Figure 1.16:

Checking is done by cellular automata

Target is to sense minimum number of bits to decide on the all 0s output

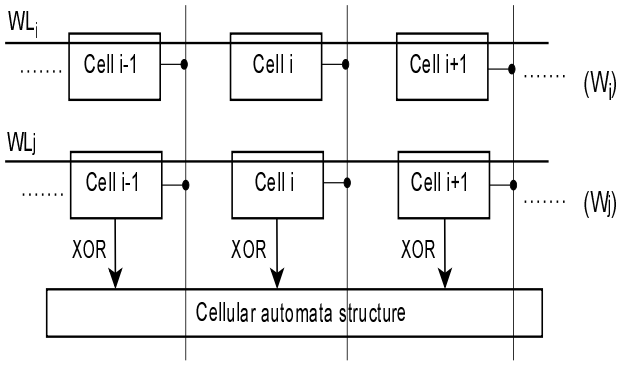
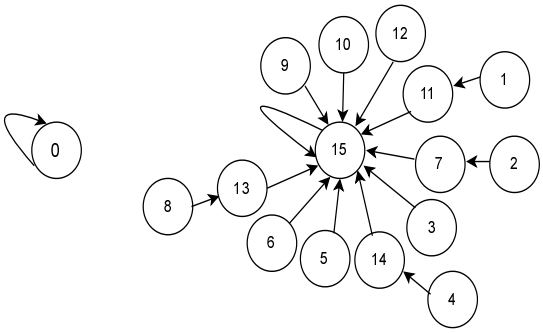
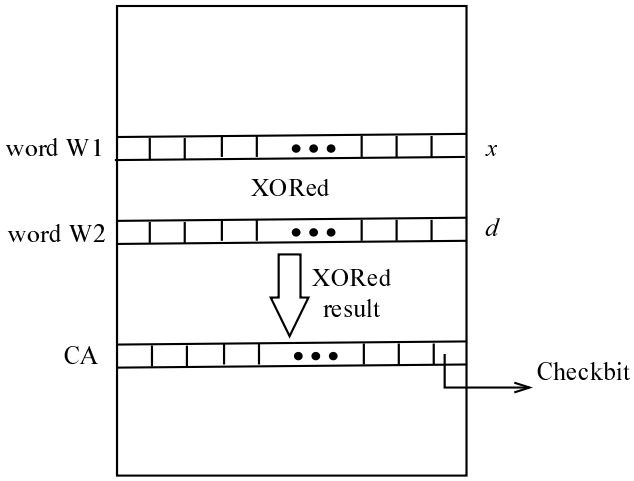


Figure 1.17:

TACA with rule 254 is employed for the In-Memory Search





LSB of TACA (checkbit) indicates match/mismatch - Decision by sensing 1 bit

In-SRAM XOR Operation

6T SRAM cell

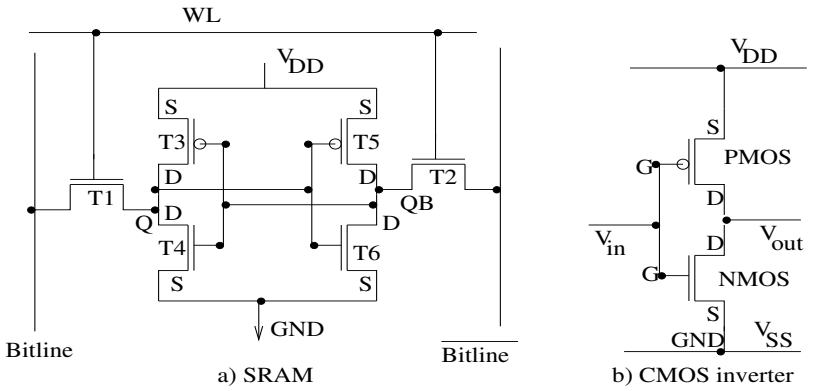


Figure 1.18: 6-T SRAM cell

Read disturb

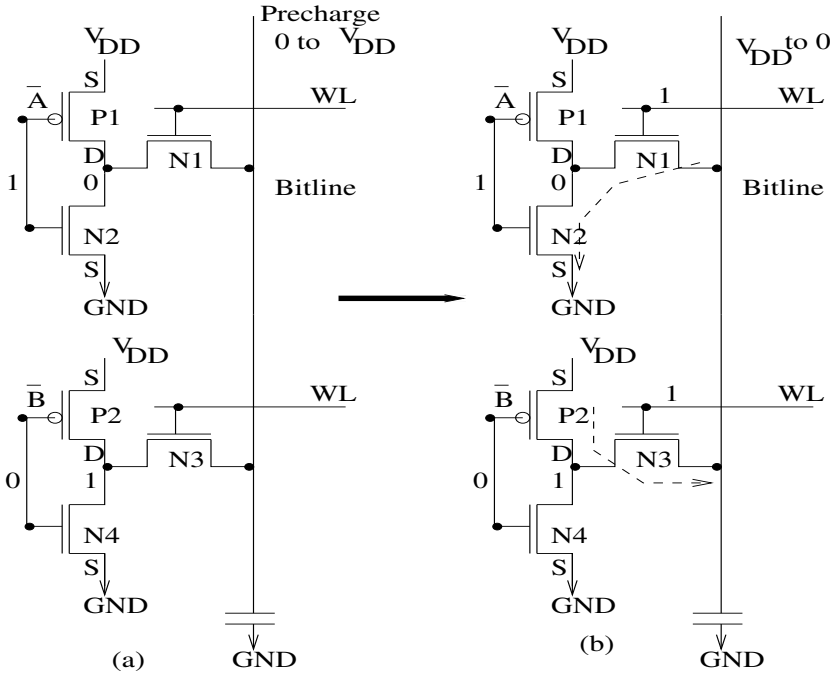


Figure 1.19: 6-T SRAM cell read disturb

8T RAM is used

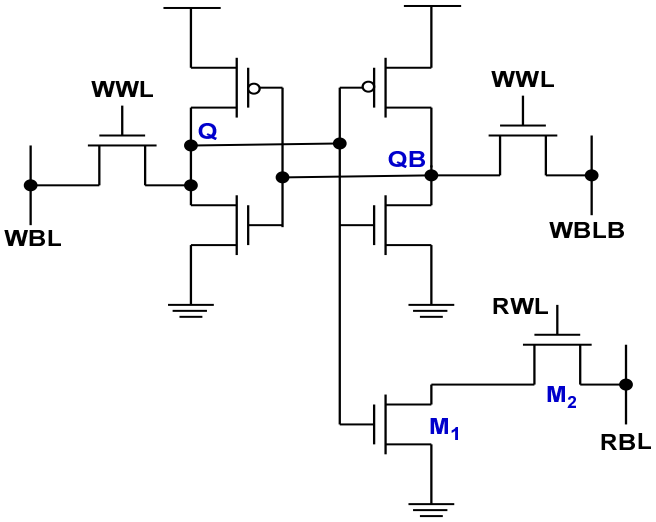


Figure 1.20: 8T SRAM

Additional transistors (M1, M2) for read, creating decoupled read ports

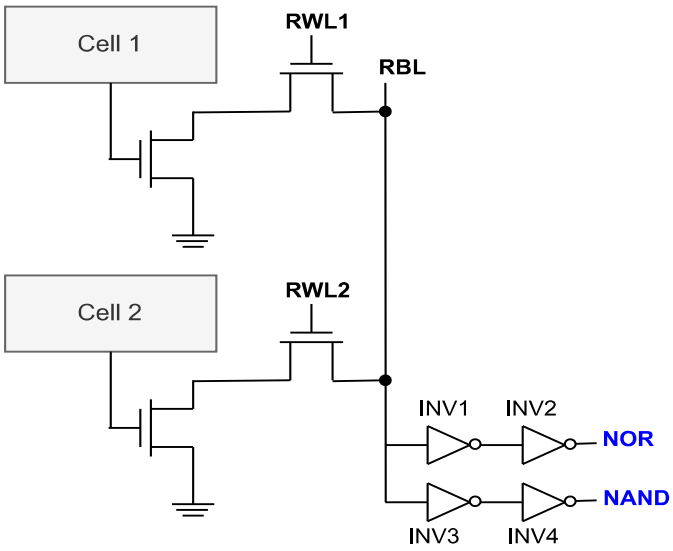


Figure 1.21: 8T NAND-NOR

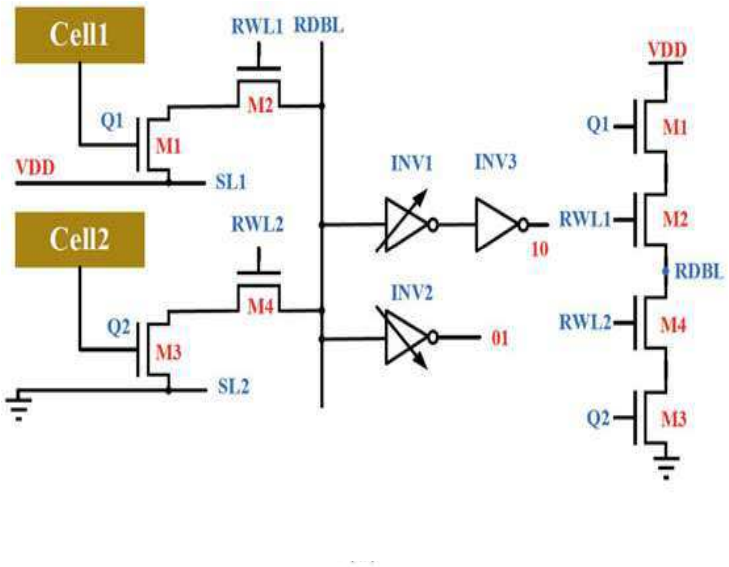


Figure 1.22: 8T XOR

SL1 is connected to VDD while SL2 is grounded, forming a voltage divider.

Both RWL1 and RWL2 are initially held at ground, and read bit-line RDBL is precharged to Vpre

Transistors M2 and M4 are then switched ON and a voltage divider is formed with RDBL

For stored values of Cell1 and Cell2 as

- 00 or 11, RDBL remains at Vpre (as M2 and M4 are OFF)
- 01, RDBL is discharged to ground (M4 is ON while M1 is OFF)
- 10, RDBL is charged to VDD (M1 is ON while M4 is OFF)

XOR output is finally obtained by sensing the RDBL using two skewed inverters

INV2 is skewed such that it goes high only when RDBL is much lower than Vpre - close to ground

INV1 is skewed such that it goes low only when RDBL is much higher than Vpre (approx VDD)

Hence by ORing (in peripheral circuit) outputs of INV2 and INV3 outcome of XOR can be get

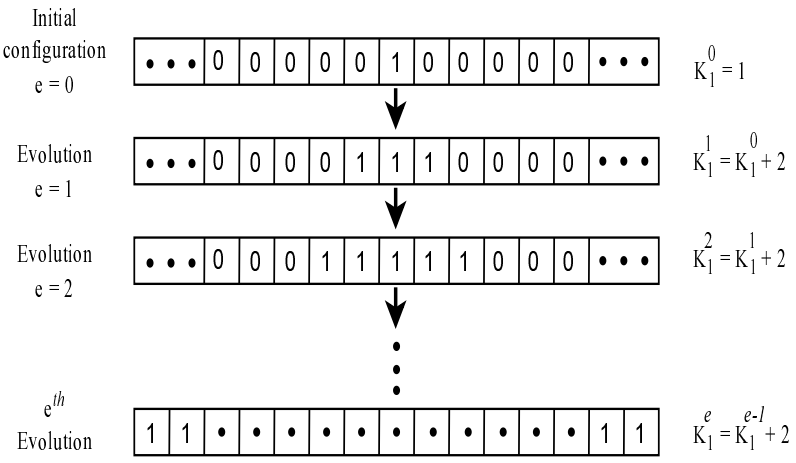

Initial configuration e = 0
Evolution e = 1
Evolution e = 2
e^th Evolution
$K_1^0 = 1$
$K_1^1 = K_1^0 + 2$
$K_1^2 = K_1^1 + 2$
$K_1^e = K_1^{e-1} + 2$

Figure 1.23:

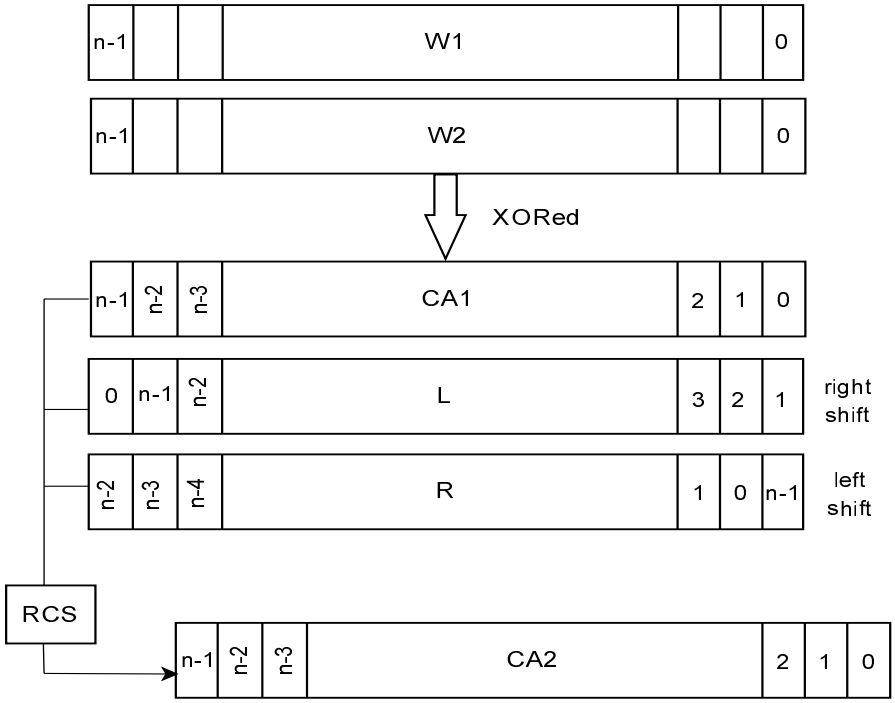


Figure 1.24:

The s, l and r on the same bitline

Rule 254 is OR operation of s, l and r -that is, OR of the three cells in a column of memory array

This is realized at skewed inverter INV1

Skewed inverter is a CMOS inverter with unequal transistor widths (PMOS and NMOS sizes)

It causes unbalanced ratios to shift the switching threshold

High-skew inverters (wider PMOS) speed up rising transitions
while low-skew inverters (wider NMOS) speed up falling transitions

The read-compute-store (RCS) scheme

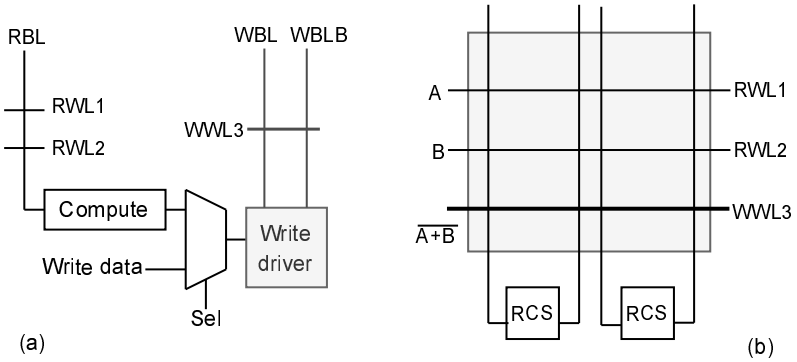


Figure 1.25:

As data is being read (source read) from RBL, for activated RWLs of two cells
simultaneously WWL of a third cell can be enabled to perform write operation (destination write)

Copy operation is also possible through RCS

It is done by activating RWL of source word row and WWL of destination word row

Data stored in bit cells of source word goes to SA

Output of SA is the input of RCS block, to be stored in destination word

Shift

Shift-Copy SRAM, is designed to achieve automatic bit alignment

Shift-Copy consists of three main components, as shown in Figure 1.26

The first is the Data SRAM

Second is the Shift-Copy SRAM, interconnected with Data SRAM via transmission gate arrays

This array serves as temporary storage

8T bitcells are used to form the Shift-Copy SRAM

Within Shift-Copy SRAM, a Shift-Copy operation has been introduced

It executes an n-bit shift when copying data from one row to its n^{th} neighboring row

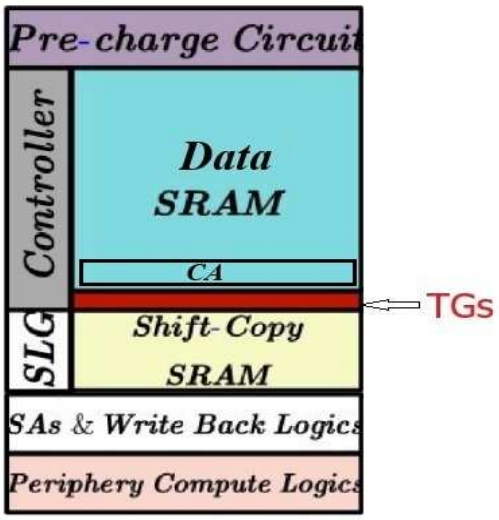


Figure 1.26: Shift-copy SRAM

Row transmission gate array (red) connects/disconnects Shift-Copy SRAM to the 6T bitcell SRAM
Figure 1.28 connects added bitlines in the 8T bitcells diagonally (blue bitlines)

It is done by substituting bitcell linked to row directly below with a bitcell shifted one bit to left (indicated by red arrows)

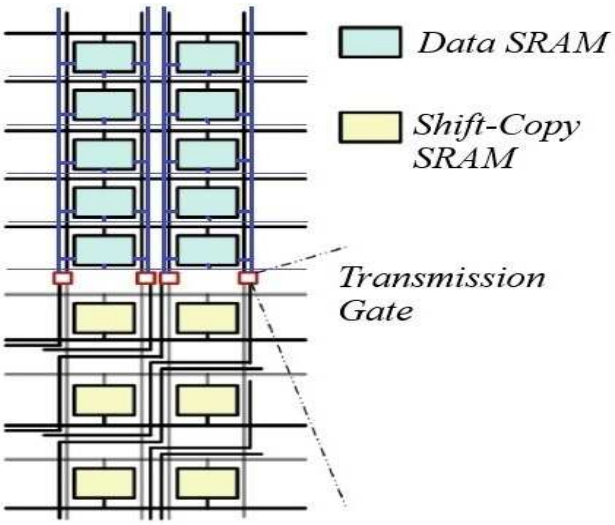


Figure 1.27: CA IMC

Figure 1.28 illustrates leftward shift of readout operation from one row to the next

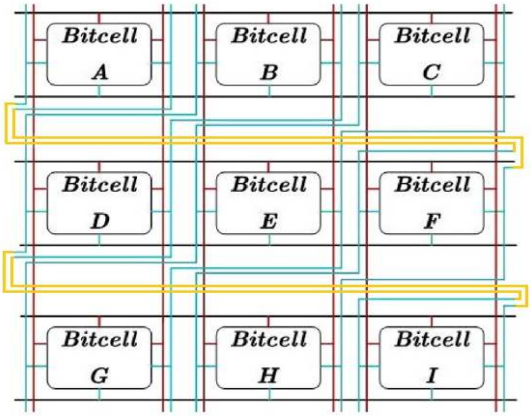


Figure 1.28: Shift copy

Let data bit 1 is stored in bitcell C

WL1 is activated to link bitcell C to OBL0 and OBLB0

Simultaneously, OBL0 is maintained at a high voltage, while OBLB0 is discharged
resulting in a slight voltage difference between the bitlines detected by SAs

Then after reading the data, it is written back using OBL0 and OBLB0

At this moment, OBL0 is precharged to a high voltage and OBLB0 is grounded

Upon activation of WL3, E is linked to bitlines, which causes bit in E (originally say 0) to be flipped

That is, write-back operation done Consequently, data bit 1 is transferred from C to E

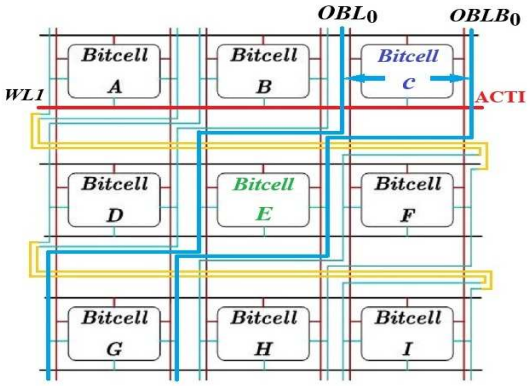


Figure 1.29: Shift copy

Finally, process of reading data from E via vertical bitlines (BL1 and BLB1) is completed

WL2, which connects bitcell E to BL1 and BLB1, is activated

Subsequently, BL1 is maintained at a high voltage, and BLB1 is discharged
resulting in a slight voltage disparity between the bitlines

This voltage variation is then detected by SA

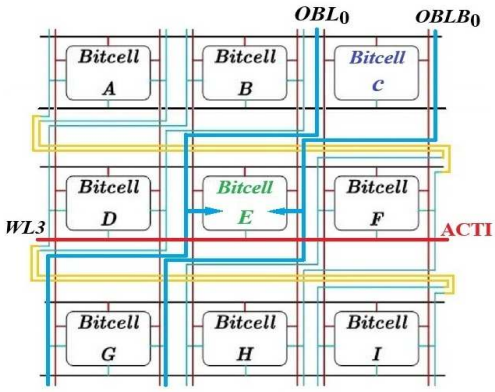


Figure 1.30: Shift copy

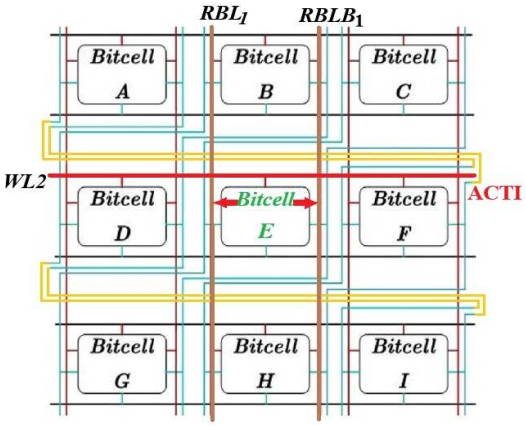


Figure 1.31: Shift copy

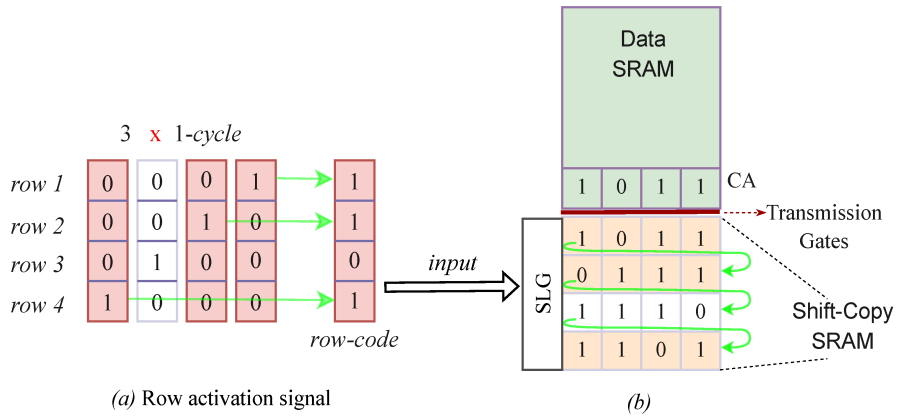


Figure 1.32: CA IMC working

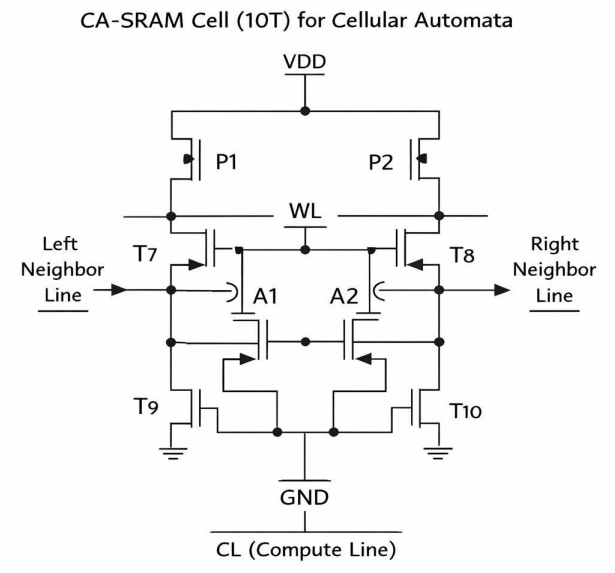


Figure 1.33: 10T left right shift

Middle of the circuit contains two cross-coupled inverters

Transistors involved: P1 and P2 $\rightarrow$ PMOS T9 and T10 $\rightarrow$ NMOS (pull-down)

Transistors A1 and A2 (access transistors) connect storage nodes to bitlines

T7 and T8 transistors are what make the SRAM cellular-automata capable

T7 $\rightarrow$ Left neighbor line T8 $\rightarrow$ Right neighbor line

At the bottom is the Compute Line (CL)

When CL is active: Neighbor signals combine at compute node

Normal SRAM mode WL = 1, CL = 0

Cellular Automata compute mode WL = 0, CL = 1

CA approach is consistent with modern notion of unified spacetime.

In computer science, space corresponds to memory and time to processing units.

In CA, memory (CA cell state) and processing unit (CA local rule) are inseparably related.

Structure of a cell is separated in two parts:

(i) Combinational one, which mainly includes all computations taking place in cells, and

(ii) Memory part, which passes combinational results to adjacent cells during next time step.

Memristive CA cell circuit implementations operate in two stages - Computation and Read stage.

During Computation Stage first three parts of the cell layout are activated
so new cell state is computed and encoded in impedance of memristive components.

It is also possible to selectively split Computation stage in two separate sub-stages -Set and Reset.

It denotes exact programming operation taking place over memristive components.

Memristive CA cells implement desired CA evolution rule and are employed to create
1-/2-dimensional, structurally-dynamic CA arrays, where CA-based algorithms are executed.

Shortest Path and Traveling Salesman Problems

There are three well-known alterations of this problem

(i) Single source shortest path -that is, shortest path from a given vertex of a graph to all others,

(ii) All pairs shortest path -that is, shortest path between all possible pairs of vertices, and

(iii) Single source single destination shortest path.

Following approaches are considered

(i) Memristor-based CA capable of detecting nodes of a given mesh belonging to shortest path from one source-node of mesh to one or multiple destination-nodes

(ii) Iterative application of memristive CA for traveling salesman problem in undirected graphs.

Basic memristive CA cell is shown in Figure 1.34.

N, E, S, and W denote the four geographical neighbors - north, east, south and west.

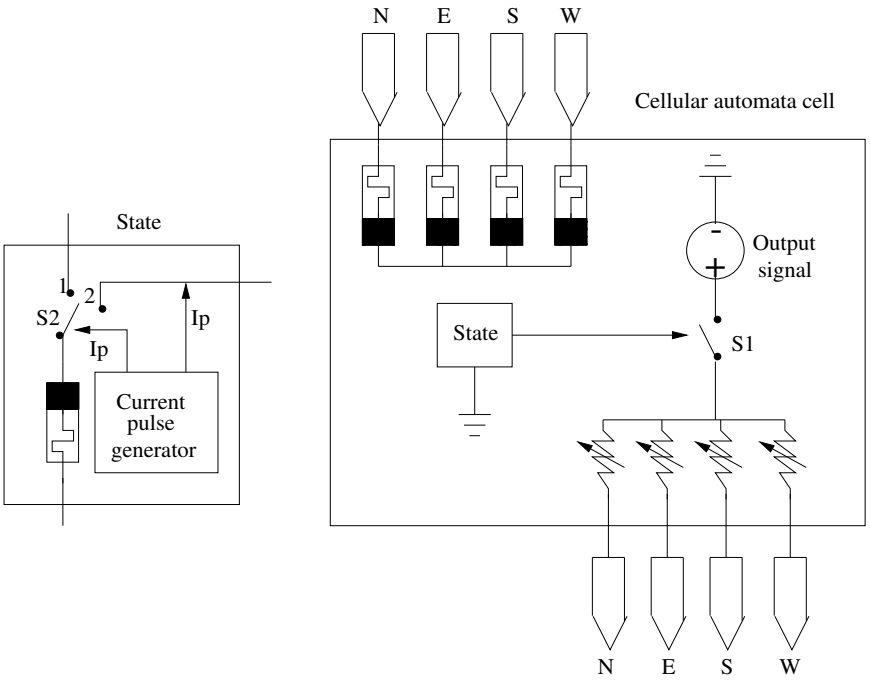
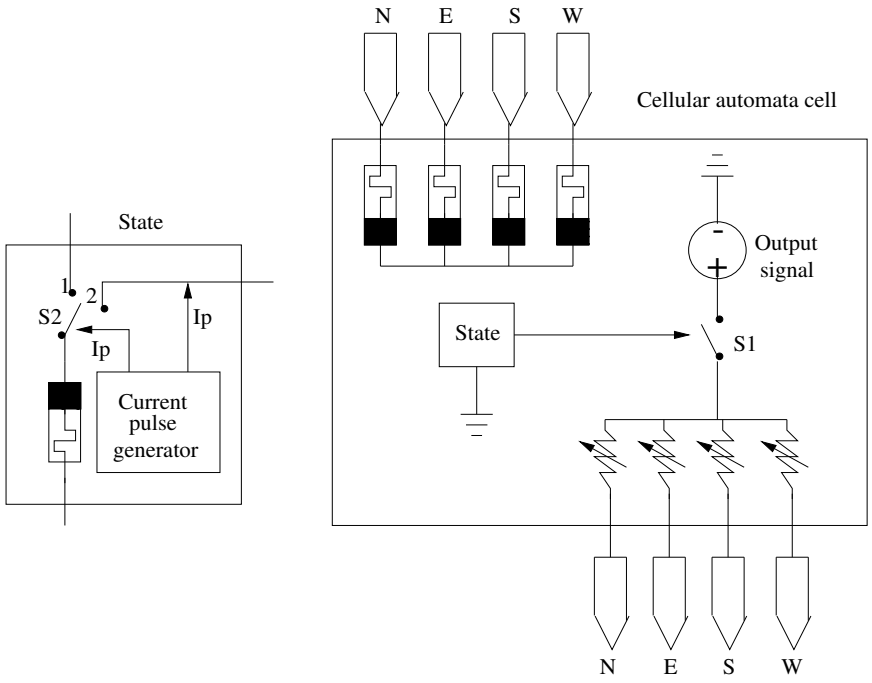


Figure 1.34: A memristive CA cell with 4 inputs and 1 output connected with nearest cells belonging to von Neumann neighborhood

Refer to forward (reversely) polarized memristors as FPMs (RPMs).

Memristance of FPM will decrease (increase) when it is forward (reversely) biased

whereas, RPM exhibits the opposite behavior.



Cell includes five memristors

an RPM holding state of cell (state-memristor), and

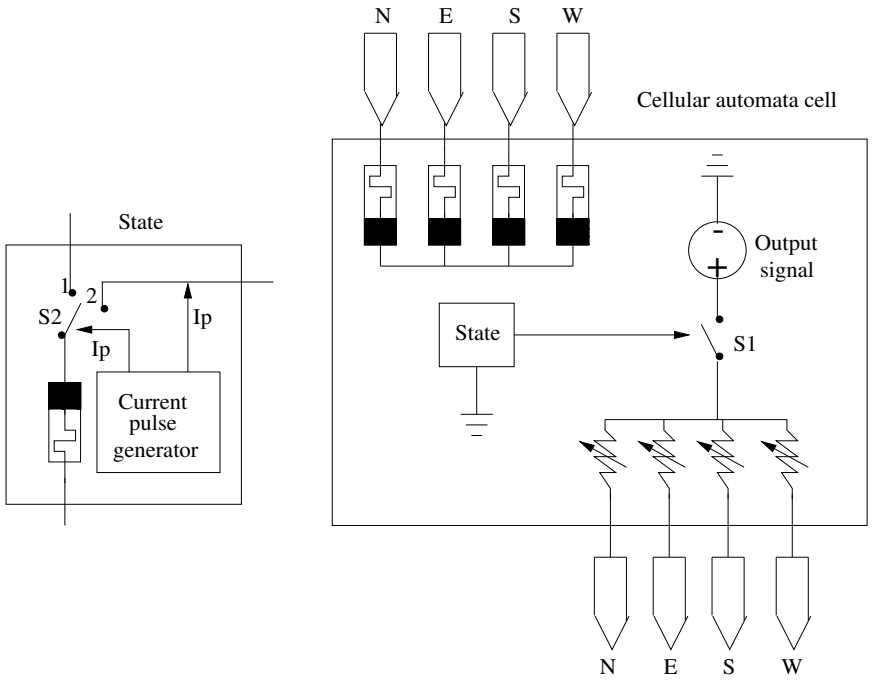
four FPMs - driven by 4 inputs corresponding to von Neuman neighborhood (input memristors).

Inputs are applied directly to memristive part of the cell.

There is simple voltage source (fixed value) - communicated to output of cell if conditions are met.

Input memristors are identical and have maximum memristance $R_{OFF} \approx 20$ kΩ

whereas, $R_{OFF} \approx 1$MΩ for state-memristor.



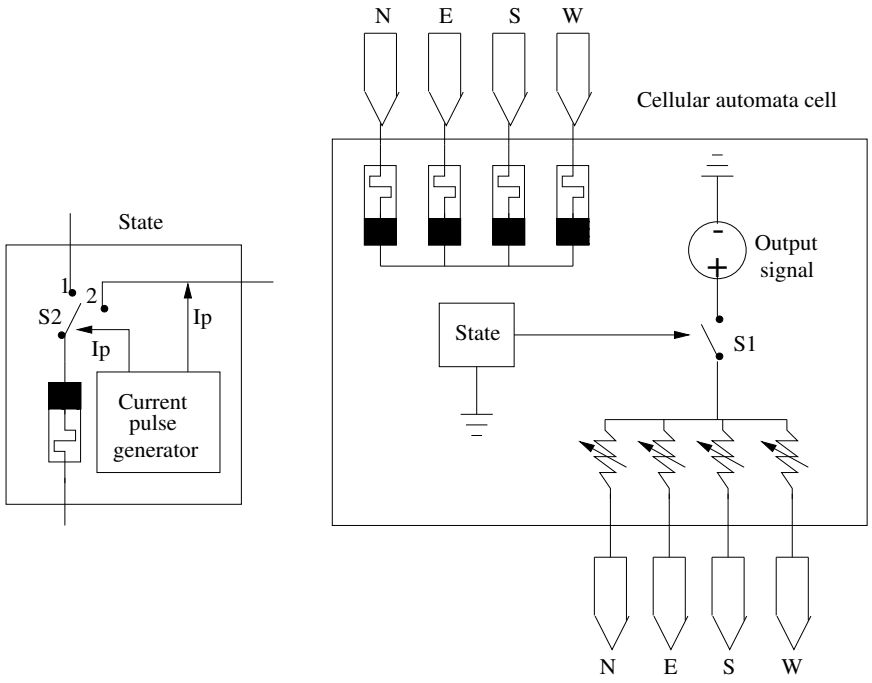
These anti-serially connected input memristors form a voltage divider circuit.

Memristance boundary values are chosen so that state-memristor, after switching to high resistance it effectively prevents state-change of input memristors which are still in high-resistance (OFF).

Therefore, there is a complementary resistive switch (CRS) only that all FPMs here are connected to a common RPM.

Cell comprises two switches, a DC voltage source, a current pulse generator (PG), and set of four variable resistors, which are used for programming intercellular connections.

Setting a variable resistor to a very high resistance (higher than that of the input memristors) will cut off communication of output to the connecting cell in corresponding direction.



All input memristors are initialized in R_{OFF} whereas, state-memristor is initialized in R_{ON}.

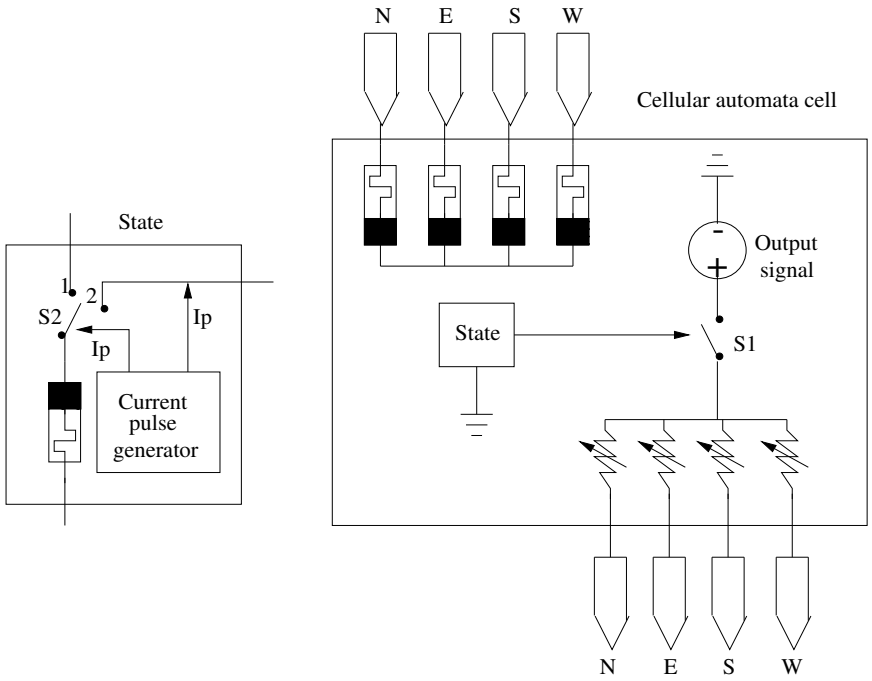
Switch S1 is initially open and the current pulse generator is driving switch S2.

Cell only receives input signals from its neighbors but does not send any signal back to them.

Links between cells can be activated or deactivated depending on states of cells connected.

Variable resistors are used to facilitate mapping of directed graphs on CA-based HW platform.

Resistors are given a low/high enough value so as to practically allow/prevent particular directed connections corresponding to directed edges of a graph.



PG drives switch S2 and provides a current pulse I_p to state-memristor in order to read its state.

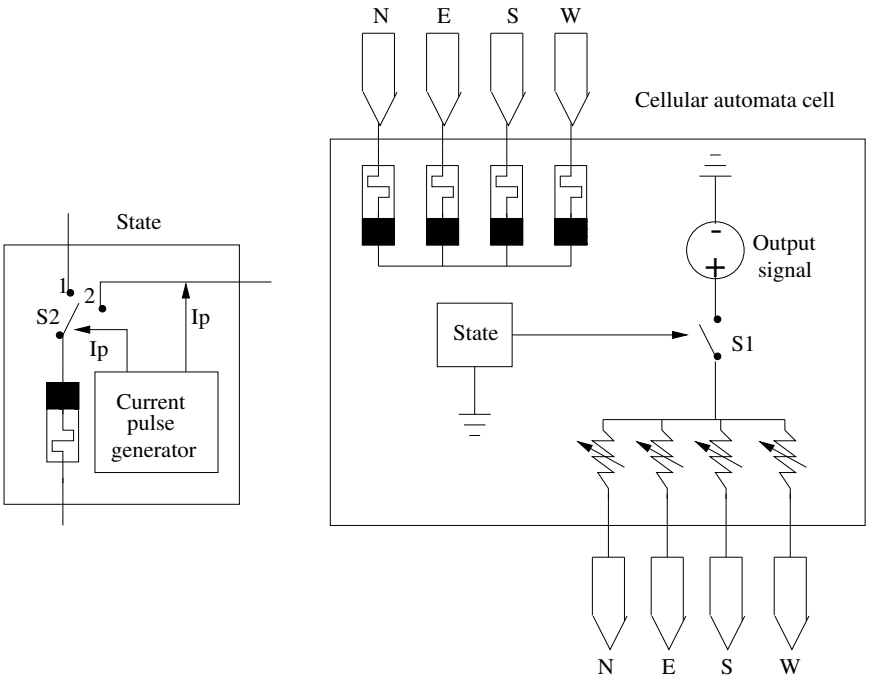
I_p consists of positive-negative paired current pulses of appropriate amplitude and duration.

Switch S2 is set to position 2 only when a positive current pulse is applied.

Negative pulse sets switch S2 to position 1 so cell updates its state while taking into account the incoming signals (if any) and previous stored state.

Whenever cell receives at least one input signal from its neighbors
corresponding input memristor(s) change their state from OFF to ON
and subsequently state-memristor switches from ON to OFF.

This change is due to complementary orientation of input and state memristors and
prevents any subsequent change to the rest of input memristors
since voltage drop on their terminals will always be below SET voltage threshold V_{SET}.



When state-memristor is being read by a short positive pulse I_p, if it is found in high-resistive (OFF)

then switch S1 closes and output ports are activated

thus allowing cell to transmit DC voltage signals to its neighbors.

Afterwards, regardless of received inputs, state of the cell cannot be further changed.

In short, all cells have two possible states

1 whenever they receive an excitation input from at least one neighbor, they switch to a different and constant state. ???

Taking account of boundary memristance values for state and input memristors, output DC voltage

is set to specific amplitude capable of causing a change to memristors of adjacent cells.

This voltage value is common in all cells regardless of size of CA.

Algorithm

In order to find shortest-path mesh grids of n vertices are arranged on a discrete lattice
where any vertex is connected to its closest neighbors by links of nonnegative weights.

The memristive CA consists of M×N rectangular array of cells $C_M(i, j)$ at coordinates (i, j)
where i = 1, 2, , M, and j = 1, 2, , N.

All cells are assumed to be identical.

For searching of shortest path between two vertices of a mesh (single source, single destination)
given graph is irst mapped onto CA.

Connections between cells are configured as unidirectional, bidirectional, or completely closed
by using variable resistors
thus facilitating mapping of any kind of directed graph whose edges are of equal weight.

Computation is initiated by triggering source cell.

Then a wave of stimulation propagates in all directions and modiies accordingly the states of cells.

Since all neighbor connections are considered to be of equal weight
aggregate weight of a path is identified by total number of hops between source and destination.

Computation is assumed finished when state-memristor of destination cell switches to OFF state or if computation steps have exceeded total number of nodes.

The later means that there is no path connecting the two given cells.

Source cell is the only one which is initialized with its state-memristor in OFF state so as to be able to transmit output signal to its neighbors from the very beginning.

In every step, switch S2 is maintained in position 1 for time t = δt

This is enough for memristors to complete their state transition when they are biased with an input.

Next, switch S2 returns to position 2 for state-memristor to be read.

When computation is completed, shortest path is found by reading state of input memristors of cells. (the reading circuit for either the input- or state-memristor is not included in Figure).

When computation is completed, shortest path is found by reading state of input memristors of cells.

Those found in ON state indicate exact neighbor(s) from where input signal was received.

These neighboring cells belong to shortest path solution.

If more than one of input memristors is found ON
    there are multiple paths reaching simultaneously to a particular cell.

In this case, all options are by default of equal total weight (hop count)
    so only one of the available paths can be randomly selected to be included in the inal result.

This way, all nodes forming shortest path(s)
    can be located by searching backwards from destination cell until source cell is reached.

Memristive CA is also suitable to find shortest paths between one source and multiple destinations.

```
initialize CA and define source and destination(s);
steps = 0;
while (no path is found) {
    t = 0;
    for all cells do {
        calculate voltage drop on memristors;
        update memristance values according to the model;
        increase t;
    } while (t ≤ δ t);
    for all cells {
        if (state-memristor is OFF) {
            close switch S1;
        }
    }
    increase steps;
    if (state-memristor of destination cell(s) is OFF) {
        shortest path found;
    }
    if (steps > total cells) {
        %% no path was found;
        exit;
        }
}
calculate backwards the shortest path(s);
```

Sorting with memristive CA

Memristor-based circuit-level parallel approach following 1-d Cellular Automata (CA).

Each cell of 1-d CA is assigned a Key to be ordered.

Within the cells, Key value is encoded in resistive state of
threshold-type switching memristors and composite memristive components.

Cell exchanges Key with either right or left neighbor.

Priority is given to the right swap in case a conflict is detected.

All cells operate using same update rule without knowing neither their index i nor length n of array.

State of each cell is defined by 3 memory elements
an integer value for Key, and two binary elements for Left/Right swapping rules SL and SR.

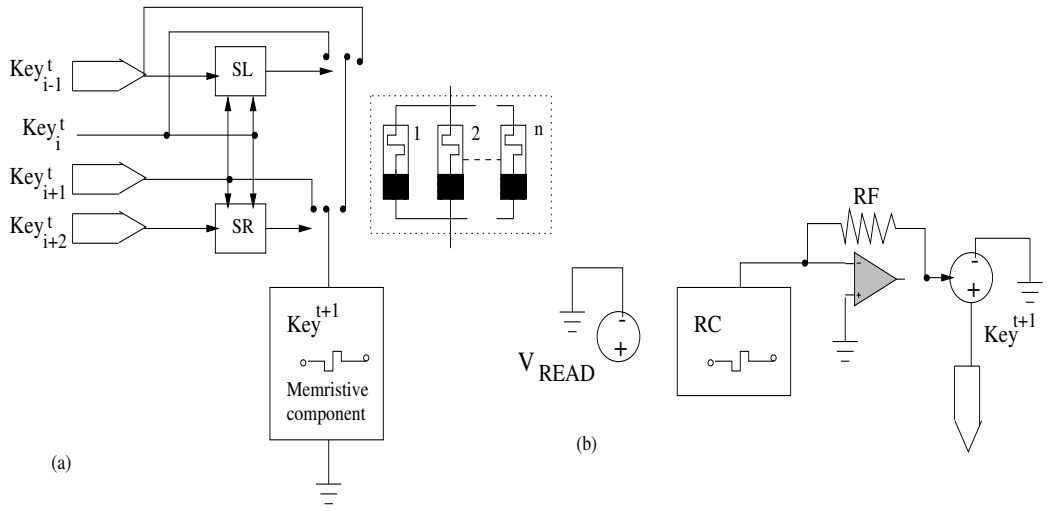
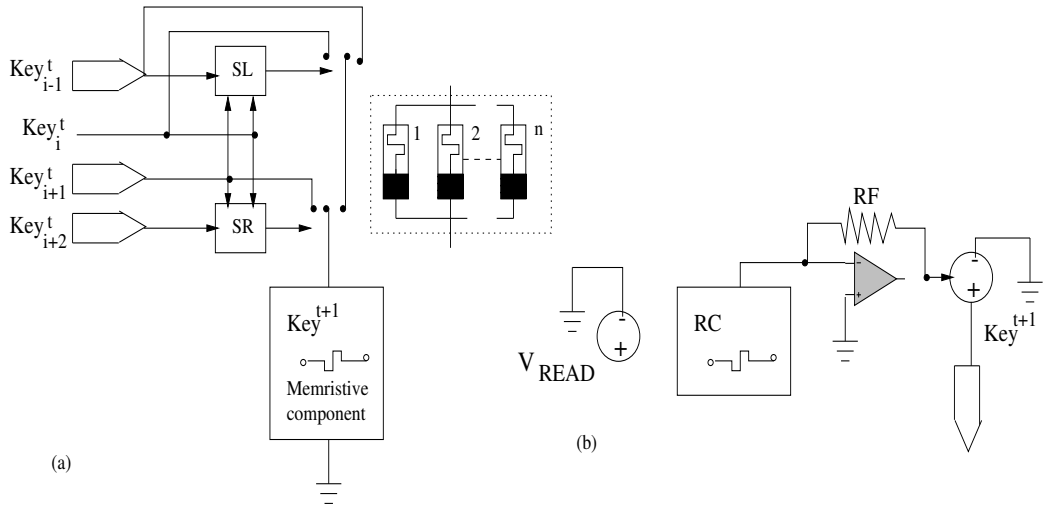


Figure 1.35: CA update rule for sorting (a) SET stage, where the memristive composite component is shown in detail. (b) READ stage

Fixed boundary conditions are applied to extreme cells of the array.



CA update rule is

$$S_L = \begin{cases} 1, & (\text{Key}_i^t \geq \text{Key}_{i+1}^t) \cap (\text{Key}_i^t > \text{Key}_{i-1}^t) \\ 0, & \text{else} \end{cases}$$

$$S_R = \begin{cases} 1, & (\text{Key}_i^t < \text{Key}_{i+1}^t) \cap (\text{Key}_{i+1}^t \; ge \; \text{Key}_{i+2}^t) \\ 0, & \text{else} \end{cases}$$

$$Key_i^{t+1} = \begin{cases} Key_{i-1}^t, & S_L = 1 \\ Key_{i+1}^t, & S_R = 1 \\ Key_i^t, & else \end{cases}$$

Every computation step comprises two stages

1. First swapping rules are computed;
2. Then Key exchanges are locally performed.

At a certain moment, array is sorted and there are no more valid exchanges to take place.

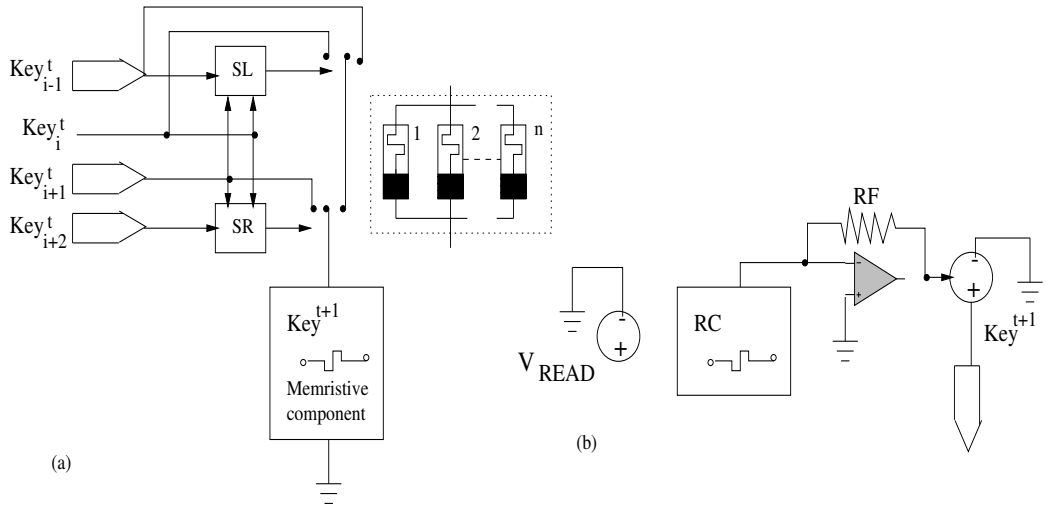
Execution can be stopped either

(i) via global control mechanism which supervises whether at least a swap has occurred in a step;

or

(ii) Determining worst-case computational time to sort the array.

Worst-case occurs when Keys are initially in inverse order and required time complexity is O($2n$-3).



CA cell circuit operation consists of 3 consecutive computational stages

RESET stage: all memristors are reset to high resistive state (R_{OFF});

SET stage: CA cells compute their next state;

READ stage: computed cell-state is stored and the cell's output is defined.

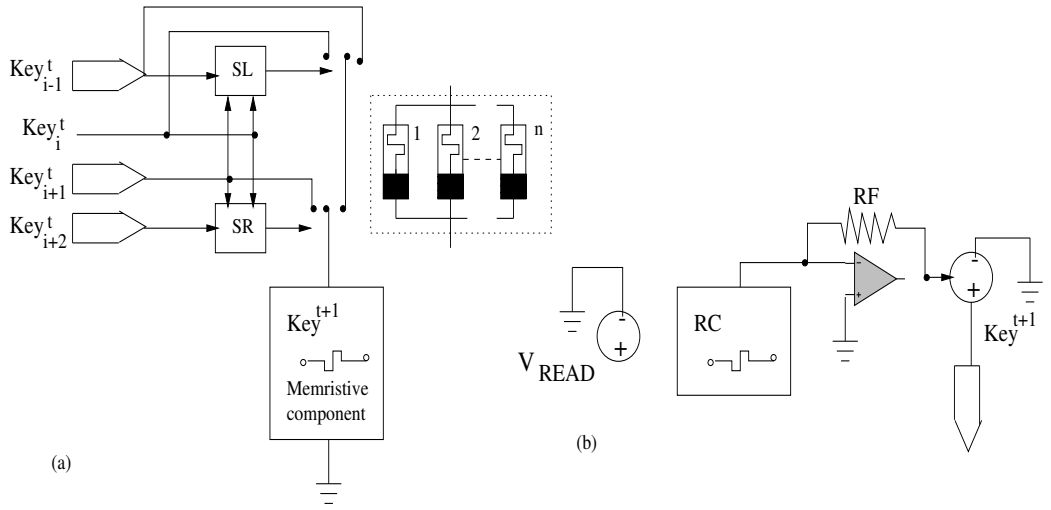
Circuit for SET stage is shown in Fig. 8.12a.

S_L and S_R define how input Keys will affect the next cell-state by controlling two switches while operating according to Eqs. 8.1a, 8.1b.

Key value is stored in the state of a composite memristive component, built according to the methodology presented in Chap. 3.

Such device consists of n parallel memristors which have different V_{SET} thresholds

$V_{SET,1} < V_{SET,2} < \cdots V_{SET,n}$, but equal memristance ranges [R_{ON}, R_{OFF}].



All memristors are formerly reset to R_{OFF} state via common negative voltage pulse.

A common negative threshold V_{RESET} for all memristors can be chosen.

So, application of a negative voltage, higher than $|V_{RESET}|$, will reset all memristors.

V_{RESET} However, under positive applied voltage composite device operates as a multi-threshold memristive device.

When voltage amplitude falls between $V_{SET,1}$ and $V_{SET,2}$, only first memristor switches to R_{ON}.

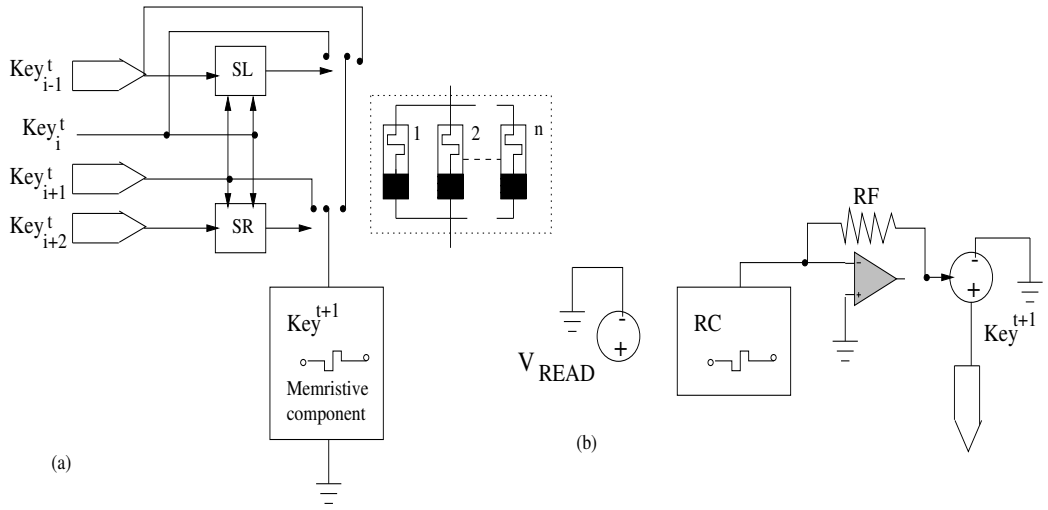
If voltage amplitude is between $V_{SET,2}$ and $V_{SET,3}$, two of the memristors switch states, and so on.

Finally, a voltage higher than $V_{SET,n}$ causes all the n memristors to switch.

The current-controlled DC voltage source is to store next cell-state

both for internal use as well as to define output of the cell.

This is better explained in Figure 1.35b - circuit which implements READ stage is demonstrated.



Resistance of composite memristive device is read by applying positive V_{READ} - low amplitude.

V_{READ} does not exceed any of the switching thresholds.

Memristors are connected to a current-to-voltage converter (I/V) of variable external gain R_F/R_C.

Resistance of feedback resistor $R_F = R_{ON}$.

R_C is variable composite memristance of parallel memristors.

I/V output is approximately given by mxV_{READ} - m is number of memristors that are in R_{ON}.

Only when all the n parallel memristors are in R_{OFF} it is

$R_C \approx (R_{OFF}/n)$ R_F and I/V output voltage becomes very small.

Based on I/V, current-controlled DC voltage is adjusted to hold next Key value (output of cell).

Example

Table 1.2

Table 1.2: Sorting with memristive cellular automata

	Cell 1			Cell 2			Cell 3			Cell 4		
t	SL1	Key1	SR1	SL2	Key2	SR2	SL3	Key3	SR3	SL4	Key4	SR4
0	0	1	0	0	2	1	1	3	0	0	3	0
1	0	1	1	1	3	0	0	2	1	1	3	0
2	0	3	0	0	1	1	1	3	0	0	2	0
3	0	3	0	0	3	0	0	1	1	1	2	0
4	0	3	0	0	3	0	0	2	0	0	1	0
5	0	3	0	0	3	0	0	2	0	0	1	0

Max Clique Problem
Bin Packing Problem
Knapsack Problem

Chapter 2

In-Memory Shift

Shift operation is needed in CA based in-memory computation, multiplication, division, etc.

There is no scope of shift operation in state-of-the-art DRAM, SRAM, Memristor array (?), QCA (?)

Shift in DRAM array

No effective solution

Shift in SRAM array

To realize shift, structure of conventional 6T/8T SRAM cell is modified

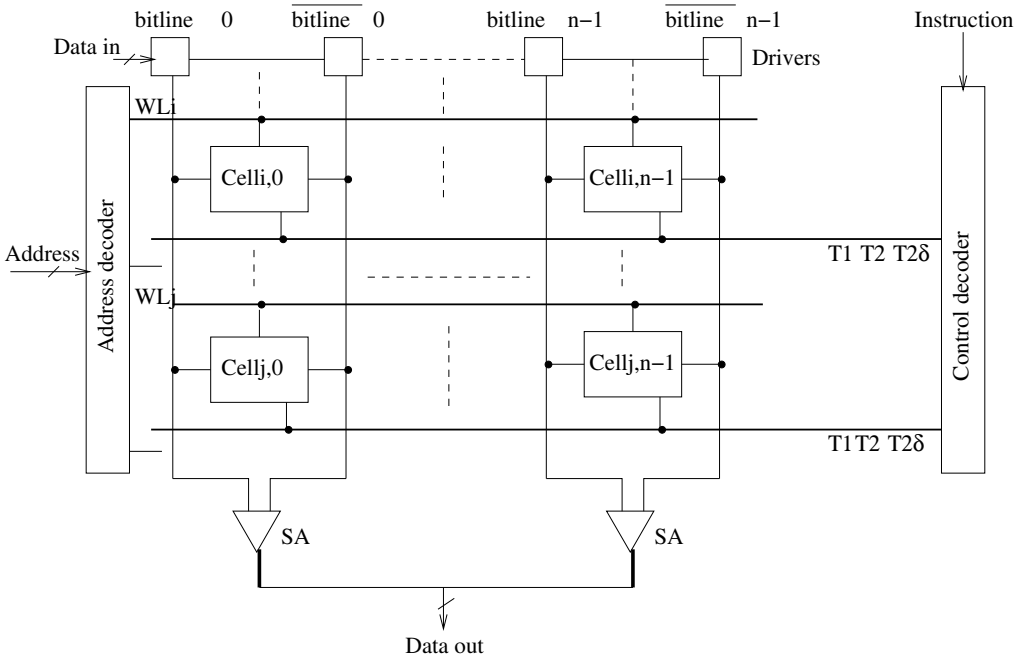


Figure 2.1: Shift SRAM

Bitline (BL) precharger and address decoder are same as those of a conventional SRAM array

Control decoder serves as an interface to the external processing units such as CPU or FPGA

SRAM cell is designed to support in-cell shift function

so that each row could be cyclically shifted independently (to the right, for example)

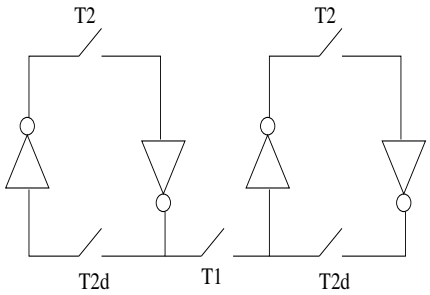


Figure 2.2: Shift SRAM cell

Each shiftable SRAM cell includes a conventional SRAM cell
a CMOS transmission gate controlled by T1 as the intercell switch
and two NMOS switches controlled by T2 and T2d as intra-cell switches

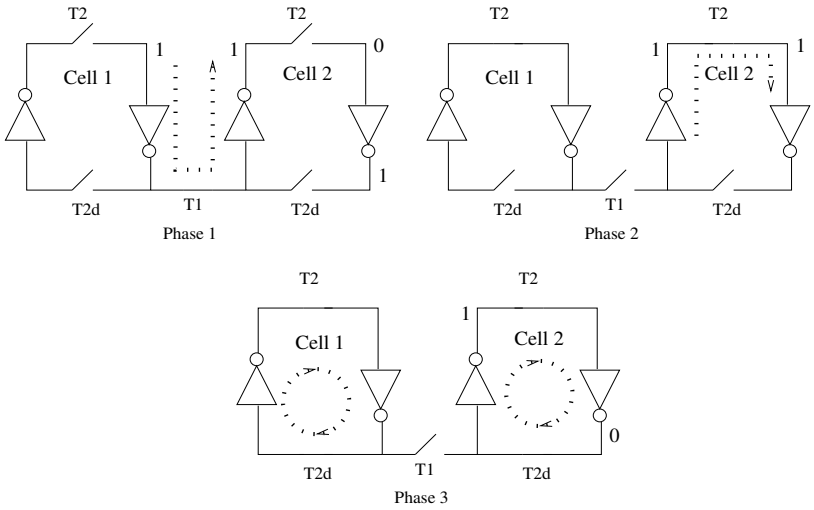


Figure 2.3: Shift phases

Shift operation between adjacent SRAM cells consists of three phases

In phase 1, intra-cell switches T2 and T2d are turned off and inter-cell switch T1 is turned on
remnant charge at Cell 1 will drive two inverters to generate a path from Cell 1 to Cell 2

In phase 2 and phase 3
intra-cell switch T2 and T2d are turned on one by one with other switches remaining off
so that each SRAM cell forms a closed loop to stabilize its datum

Control signal T2d is set to T2 with a slight delay to provide time for data restoration in phase 2

The delay circuit could be simply realized by two serially connected inverters.

REALIZE LEFT SHIFT ACCORDINGLY
REALIZE RIGHT SHIFT FOLLOWING OTHER METHOD
COMPARE THE DESIGNS

Alternative design

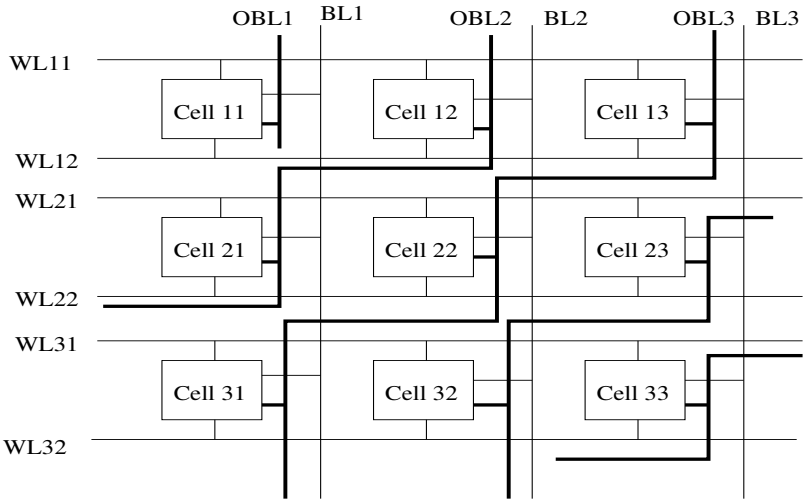


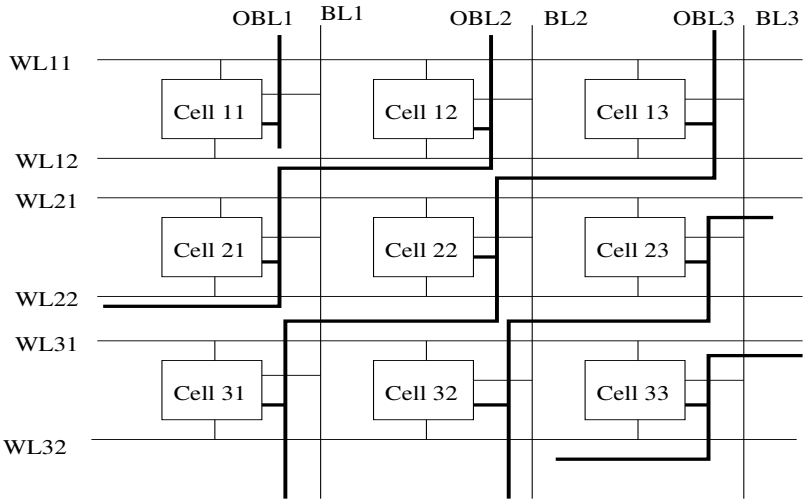
Figure 2.4: Shift copy SRAM

Employs 8T SRAM cells. Bitlinebars are not shown to avoid complexity

Shown left shift
There are oblique bitlines (OBL/OBBL) that are connected diagonally (bold bitlines)

Connection made by substituting bitcell linked to row directly below with a bitcell shifted 1-bit left

Process of executing Shift-Copy operation



Let data bit 1 is stored in C13

Precharge OBL3 (OBLB3, not shown)

WL12 is activated to link bitcell C to OBL3 (as well as OBLB3, not shown)

OBL3 is maintained at a high, while OBLB3 is discharged

Resulting in a slight voltage difference between bitlines detected by SA - it is a data read

It is then written back using OBL3 (and OBLB3)

That is, OBL3 is precharged to a high (and OBLB3) is grounded and WL22 = 1

Bitcell C22 is now ’1’

Finally, reading of data from bitcell C22 via vertical bitlines (BL2 and BLB2)

- It is done by setting WL21 = 1 (that connects C22 to BL2 and BLB2)
- BL2 is maintained at a high voltage, and BLB2 is discharged
- Resulting in a slight voltage disparity between the bitlines
- This voltage variation is then detected by SA

To solve read disturbance short WL+BL boost is adopted

Circuit-level parallelism in Shift-Copy SRAM

Shift-Copy can be divided into two parts

read-out (shown in Figure 2.5(a)) and write-back (shown in Figure 2.5(b))

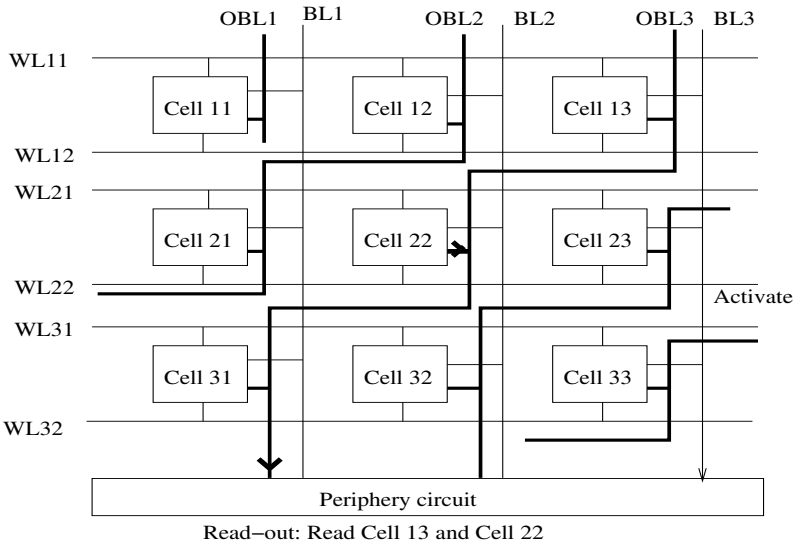


Figure 2.5: Micro-operation in Shift-Copy SRAM

Bitcell structure in oblique

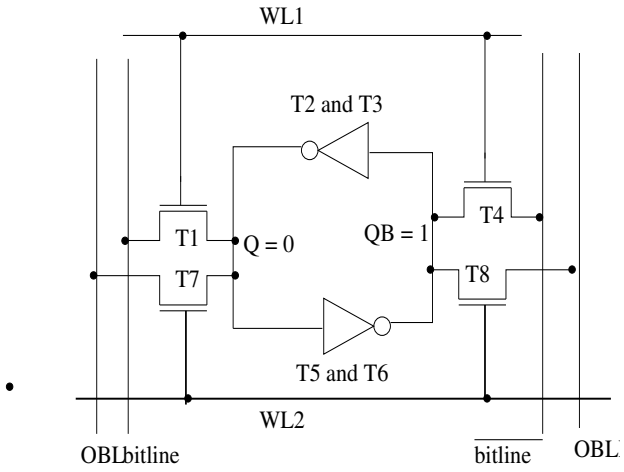


Figure 2.6: Cell structure in Shift-Copy SRAM

Shift in Memristor array

Fused RRAM-Based Shift-Add Architecture for Efficient ...

Shift in QCA array

Chapter 3

QCA - Akers for In-memory Computing

Quantum cellular automata (QCA) is a new technology - replaces current flow as information carrier by coulombic interactions of electrons within confined configurations.

QCA offers a novel alternative to the transistor paradigm.

A QCA cell (Figure 3.1) is comprised of four sites, two of which can be inhabited by electrons.

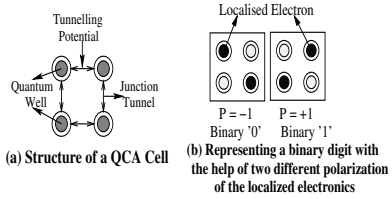


Figure 3.1: A qca cell

Coulombic repulsion between electrons will drive them to opposite corners of the cell.

State of a cell is represented by the configuration of electrons within.

Coulombic interaction between neighboring cells can be used to propagate configurations.

This forms a wire.

Coulombic interaction causes electrons to move within a cell, not between cells - no current flow.

This avoids heat dissipation and power consumption problems of transistor computing.

Unlike CMOS, in QCA transmission media and logical elements are comprised of same block - cell.

As such, QCA has been called processing-in-wire.

To avoid loss of coherence in QCA circuits, a four phase clocking system has been developed.

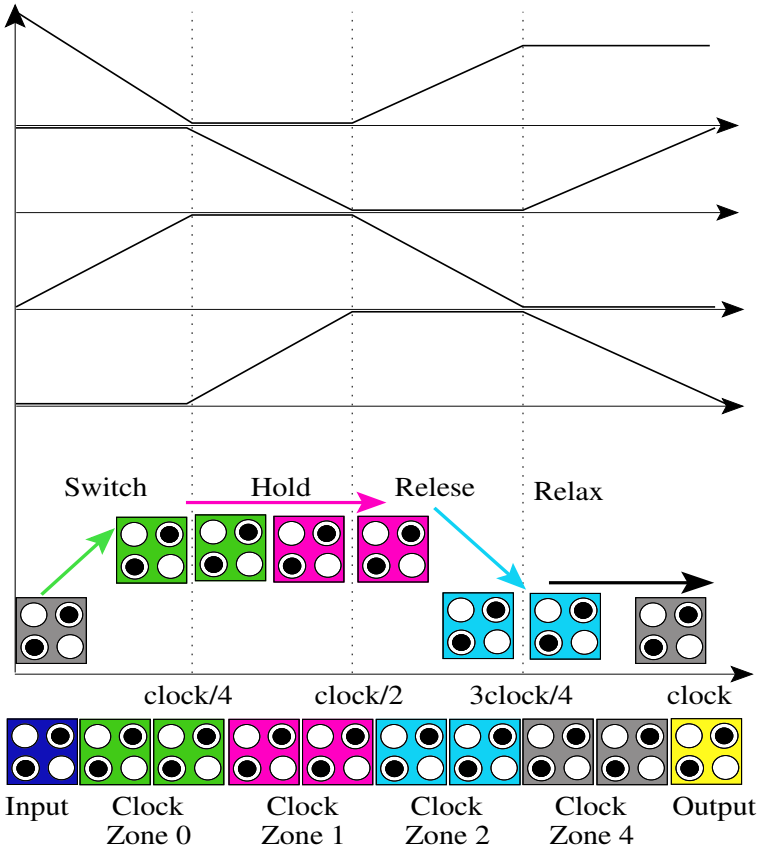


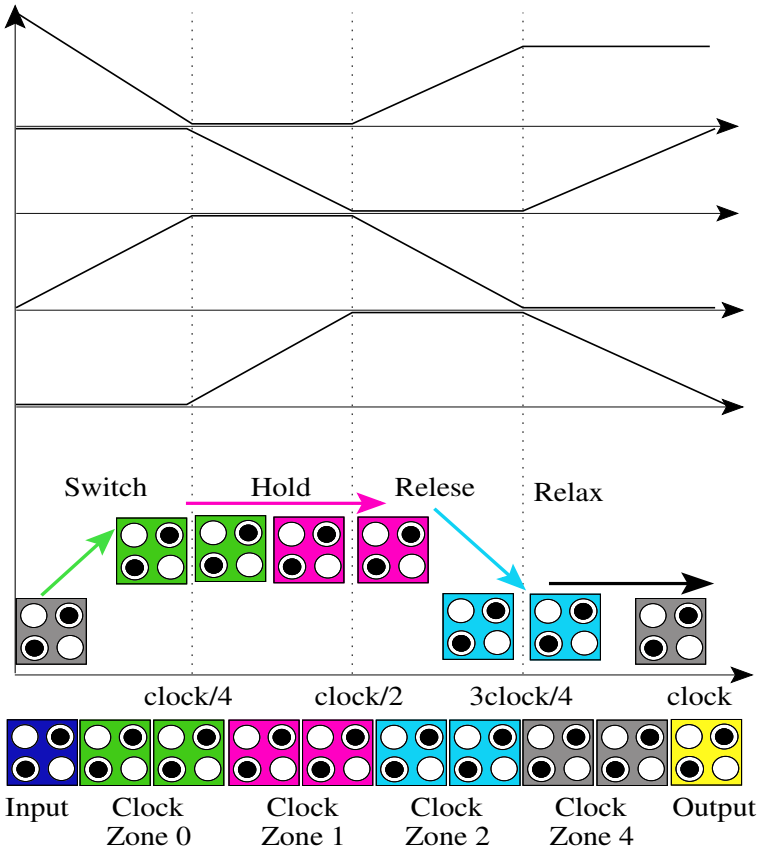
Figure 3.2: QCA clocking

An electric field is applied to the cells.

This field raises or lowers tunneling barriers between electron sites within a QCA cell.

Cells can be grouped into zones so that the field influencing all the cells in a zone will be same.

A zone cycles through 4 phases.



In Switch phase, the tunneling barriers in a zone are raised.

Then electrons within cell can be influenced by Coulombic charges of neighboring zones.

Zones in Hold phase have high tunneling barrier - cannot change state - can influence adjacent zones.

Release and Relax decrease tunneling barrier so that the zone will not influence other zones.

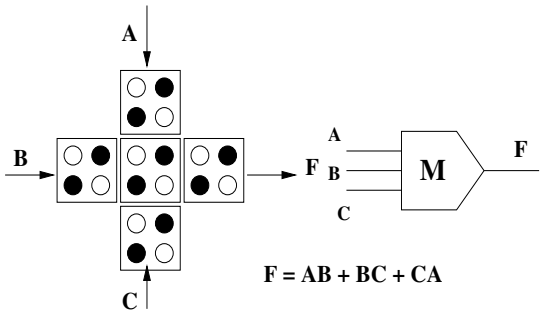


Figure 3.3: QCA majority voter

5-input minority gate-based CAM cell

Polarization of QCA cell - determined by Columbic interaction between cells - called kink energy.

It is computed using electrostatic interaction between all electrons in two neighboring cells i & j as

$E^{i,j} = \frac{1}{4\Pi\epsilon_0\epsilon_r} \sum_{n=1}^{4} \frac{q_n^i q_m^j}{|r_n^i - r_m^j|}$

where ϵ_0 is permittivity of free space.

ϵ_r is the dielectric constant

q_n^i is the charge in dot n of cell i

r_n^i is the position of dot n in cell i and $|r_n^i - r_m^j|$ is distance between cells.

Kink energy is the difference in energy between those have opposite polarizations and those have same polarization:

$E_{kink}^{n,m} = E_{P_n \neq P_m}^{n,m} - E_{P_n = P_m}^{n,m}$

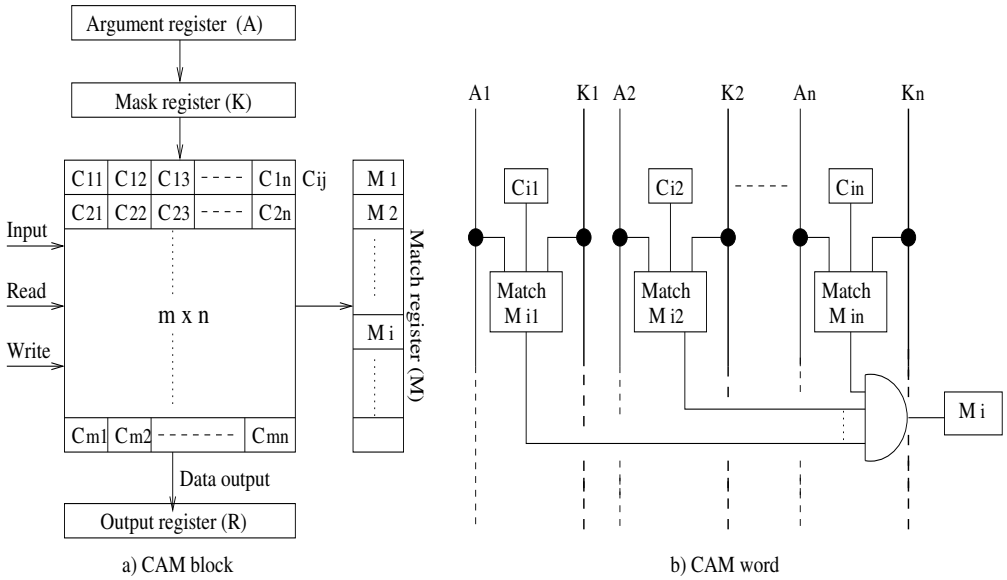


Figure 3.4: CAM block

QCA-PIM

Combination of Akers array and QCA helps to achieve high-speed computing at nanoscale.

One way to create a PIM architecture is to combine the Akers logic array and QCA.

A rectangular Akers logic array is shown in Figure 3.5.

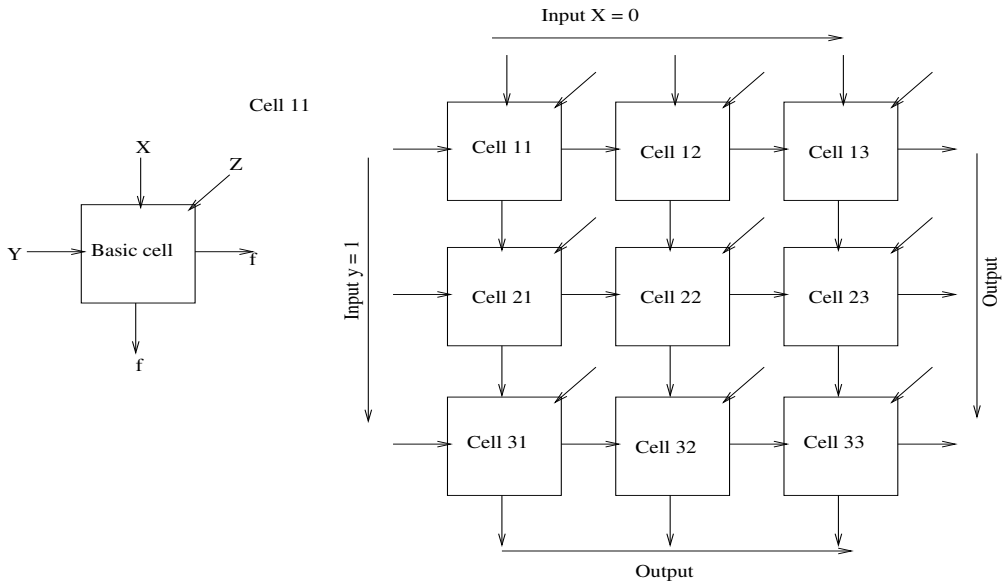
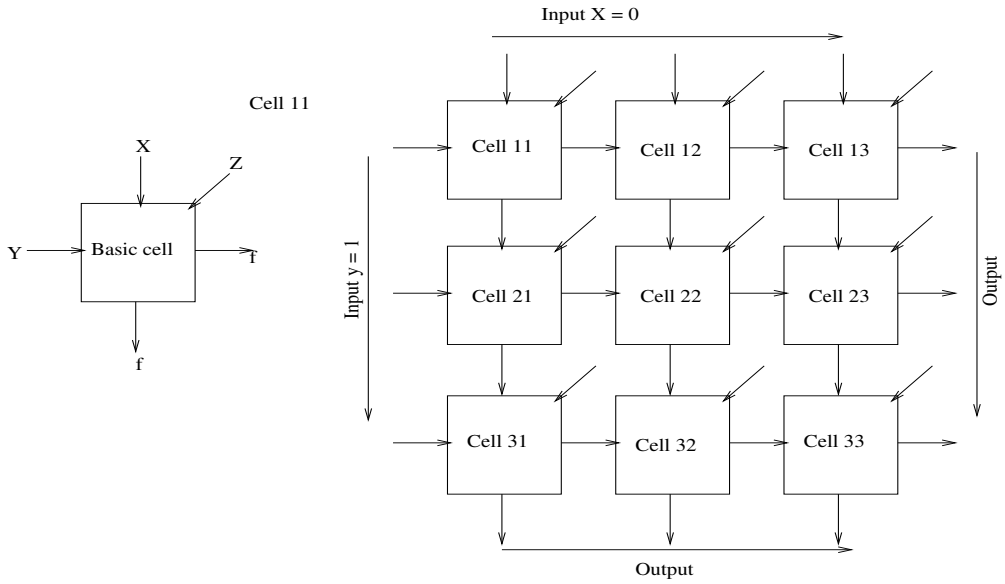


Figure 3.5: 3×3 Akers array

Akers array was introduced by mathematician S. B. Akers in 1972.

Array is composed of identical logic cells interconnected in a rectangular grid.



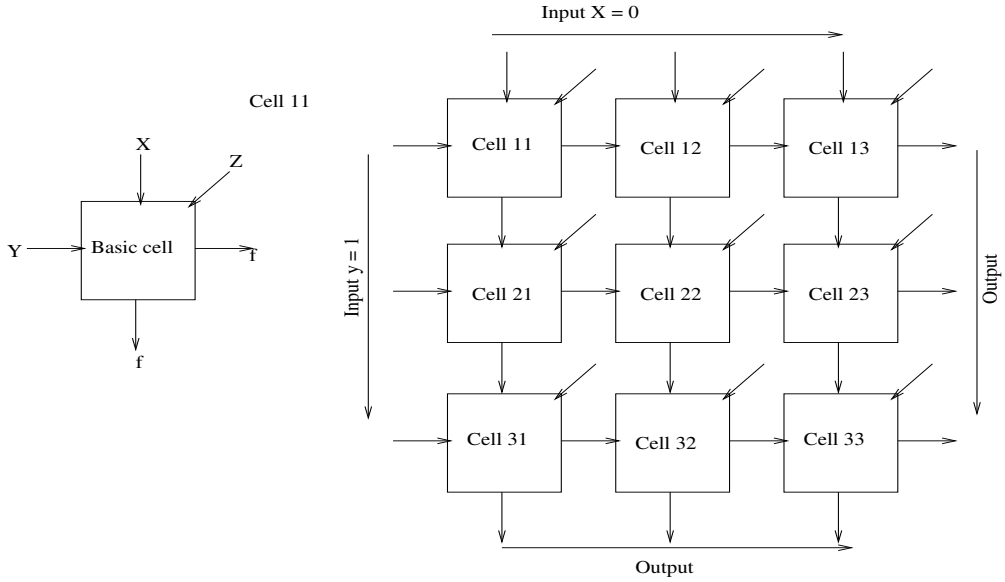
Each cell has three inputs and two outputs.

Two of inputs are fixed connections - one immediately above and one to the cell immediately to left.

Two output leads are permanently connected to two cells immediate right and immediate below.

Third input to cell is an external input on which an arbitrary input variable may be introduced.

This will be assumed to be the only input or output under designer's control.



Synthesis problem to be considered becomes that of introducing input variables to proper cells.

It enables realization of a given combinational switching function.

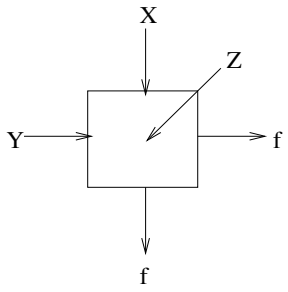
One way to visualize this structure as a single two-dimensional ”logic layer”
with external inputs being inserted to each cell from a second ”input layer” directly above.

By defining logical operation of cells in proper way - several logic functions can be realized.

It is to be ensured that certain input combinations to the cells will never occur.

This has immediate effect of don’t care
which in turns permit any of several logic functions to describe cell’s operation.

Example: Consider the cell shown in Figure 3.6.



X	Y	Z	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	–
1	0	1	–
1	1	0	1
1	1	1	1

Figure 3.6: Akers cell

Assume Akers array behaves as a 3-input majority gate but never receives the input combinations
X = 1, Y = 0, Z = 0 or X = 1, Y = 0 Z = 1.

Then resulting truth table is as shown in Figure 3.6.

'-' in column f denotes don't care.

Now, if it can be ensured that an input combination X = 1 and Y = 0 will never occur
then there are four switching functions that precisely describe a cell's operation.

These are obtained by filling the blanks with 00, 01, 10, and 11, respectively.

These functions are

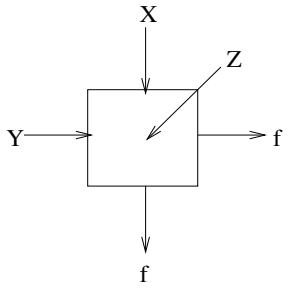
$f_0(X, Y, Z) = XY + YZ.$

$f_1(X, Y, Z) = M(X, Y, Z) = XY + YZ + ZX.$

$f_2(X, Y, Z) = XZ' + YZ.$

$f_3(X, Y, Z) = X + YZ.$

These four alternative equations, which can be used to realize PIM structures.



X	Y	Z	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	–
1	0	1	–
1	1	0	1
1	1	1	1

The basic function of the array is

f_3(X, Y, Z) = XZ' + YZ.

Input Z is seen as the value stored in cell (similar to CA but output is propagated to different cells).

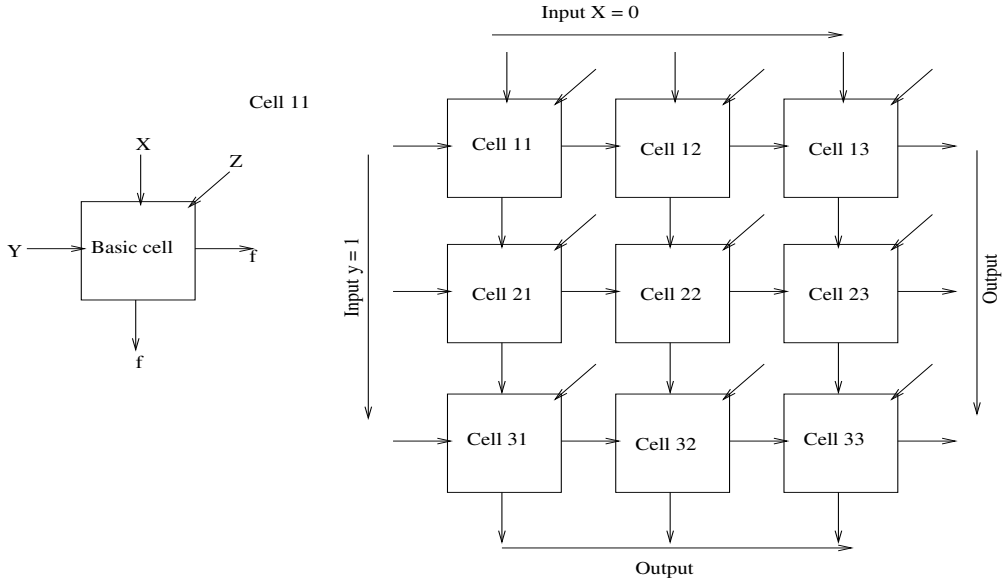
The array size can be $m \times n$.

Akers showed that two inputs X, Y to every cell always satisfy the property that X $\leq$ Y.

Let check how it can be ensured that X = 1, Y = 0 condition will never occur on inputs to a cell.

Let check how it can be ensured that X = 1, Y = 0 condition will never occur on inputs to a cell.

That is, for any cell X $\leq$ Y.



All that needs to be done is to tie all the X inputs in the top row to 0 and all the Y inputs in the left column to 1.

(Note that none of the external Z inputs are constrained.)

Now it is easily seen that the X $\leq$ Y condition will be satisfied for all cells in the top row (since X = O) and for all cells in the left column (since Y = 1).

Moreover, it follows that for each of these cells an even stronger relationship holds. Namely, X $\leq f \leq$ Y.

It can be shown that this stronger relationship holds for any cell.

The argument proceeds inductively.

Assume for a given cell C, the condition has been hold for the cell immediately above (A) and the cell immediately to the left (B) (Figure 3.7).

Then, by hypothesis X $\leq f'$ and $f' \geq$ Y.

Thus, X $\leq$ Y and, from the truth table for the cell, X $\leq f \leq$ Y.

Thus, this cell and hence all of the cells must satisfy the X $\leq f \leq$ Y condition.

Once it is decided that all the X and Y inputs are permanently fixed to 0 and 1 or tied to neighboring cells, we omit these inputs from the diagram and simply show the Z inputs as matrix (Figure 3.8).

The function f_2(X, Y, Z) = XZ' + YZ is definitely of interest. For one thing, it is precisely the Shannon function that is used to expand a Boolean function about a given input variable,

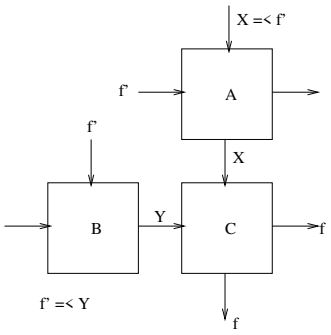


Figure 3.7: Akers relation

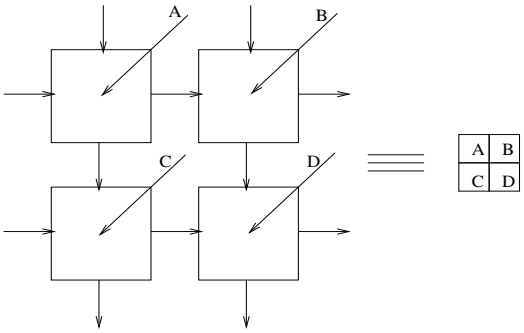


Figure 3.8: Akers input matrix

(e.g., $f(\mathrm{X}_1, \mathrm{X}_2, \cdots, \mathrm{X}_n) = \mathrm{X}_1' f(0, \mathrm{X}_2, \cdots, \mathrm{X}_n) + \mathrm{X}_1 f(1, \mathrm{X}_2, \cdots, \mathrm{X}_n)$).

Also, if we set Z = 0, then f_2 reduces to X, while with Z = 1, f_2 becomes Y.

Thus by tying Z to 0 or to 1, a cell can be ”shorted” either vertically or horizontally.

This property proves quite useful in following the signal flow through an array, and also for detecting and isolating defective cells.

One of the goals of Akers by representing his logic arrays was to show that every boolean function can be calculated us- ing this method. Even though the answer is affirmative, the number of needed cells for this task can be exponential in the number of variables.

In the Akers array, the X and Y inputs are set, respectively, to zero and one to satisfy $X \leq Y$.

The data are then transmitted to the horizontal and vertical neighbors in the logic array.

Boolean operations are performed, especially by arranging array cells side by side.

The output is received by the lower right cell of the array.

(or from multiple cell outputs in the case of a Boolean function with a multiple bit output or, alternatively, multiple Boolean functions simultaneously computed within the same array).

Hence, the same array can be used for different Boolean functions, each specifying a different organization of inputs.

The control Z stores the data somewhere and performs the entire array of Boolean operations.

Hence, the Akers array can be considered as the computational structure of PIM.

Figure 3.9 shows computation in Akers array.

Each Akers cell acts as a majority gate.

The usual way to calculate the output of an Akers array is to start from the cell in the upper left corner and continue the calculation sequentially.

Assume that the external input for the cell on the upper left is 1, and this cell receives 1 from the left, so its output becomes 1.

Now, consider the lower cell receives 1 from the upper cell and 1 from the left, so its output is 1 regardless of its other input.

Similarly, the above process is repeated for all cells in the lower left column.

Therefore, the external input 1 to the cell in the upper left causes the output of all cells in the left column to be 1.

In fact, the left column can be removed from the main matrix to create a smaller matrix.

By a similar argument, it can be shown that when the input value of the upper-left cell is 0, the top row of the main matrix is removed.

Therefore, to calculate the output of the Akers array, the rows and columns must be deleted due to the above rules to move to the last cell on the lower right and determine the final output.

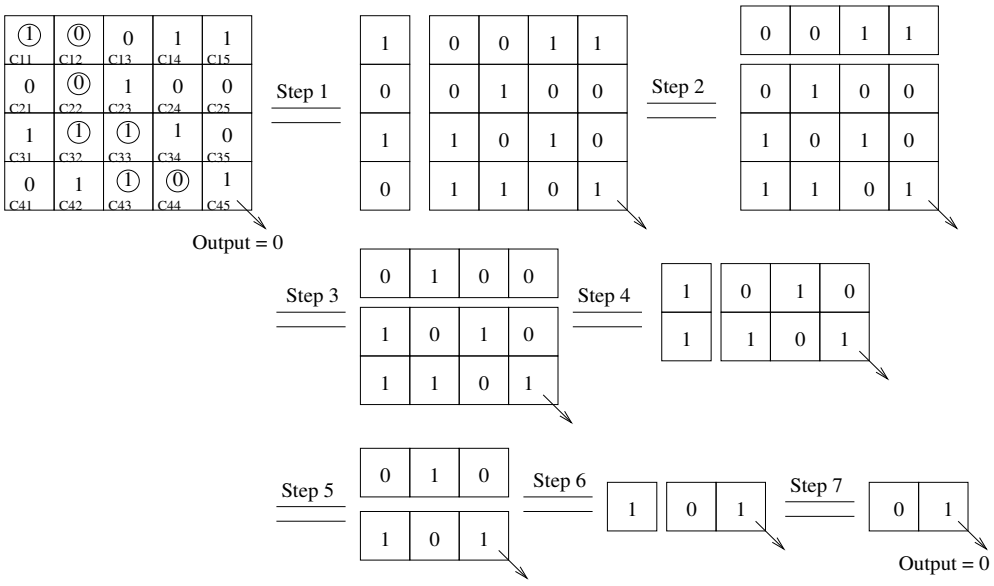


Figure 3.9: Stripping process to calculate output of Akers array

Each cell of Figure 3.9 behaves like 3-input majority gate

Start with the the upper left corner (all of whose inputs are known) and successively calculate each of the individual cell outputs until the final output cell is reached.

However, a much quicker method is available.

Assume (as in this example) that the upper left cell has an external input of 1.

Then since it also receives a 1 from the left, its output C11 will be 1.

Similarly, the outputs of C21, C31 and C41 are 1 regardless of the external input of such a cell.

Thus, an external input of I into the upper left cell forces all cells in the left-hand column to have outputs of 1.

This is precisely the case that would result if the left-hand column were deleted from the original matrix and the smaller matrix considered (shown after Step 2.

1- AND 0-PATHS

The foregoing procedure reveals that wnenever the output of an $m \times n$ array is 0, there must exist a 0 in each of the m rows of the input.

Moreover, 0 in row i must be at least as far to the right as the 0 in row i-1.

Such a set of 0s is called a 0-$path$

That is, given an $m \times n$ binary input matrix Z_{ij} a set of m 0 inputs

$Z_{1j_1}, Z_{2j_2}, \cdots, Z_{mj_m}$,

will be called 0-$path$ if and only if

$j_1 \leq j_2 \leq \cdots j_m$ (Figure 3.10).

Likewise, an 1-path in an input matrix will be a set of n 1-inputs,

$Z_{i_1 1}, Z_{i_2 2}, \cdots, Z_{i_n n}$,

having $i_1 \leq i_2 \leq \cdots i_n$ (Figure 3.11).

Note that in any binary matrix one and only one of these two types of paths will exist.

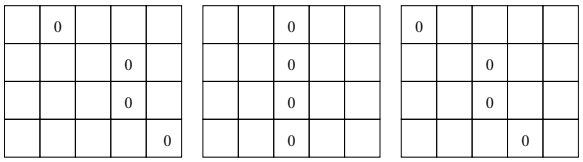


Figure 3.10: Akers 0-path

By using Akers arrays, different logic operations (such as AND, OR, NAND, and NOR) and other circuits can be designed.

Akers arrays are known for their ability to perform in-memory computations.

More efficient structures can be designed in QCA technology using systolic arrays.

a PIM structure was used to design an EXOR gate in binary QCA tech- nology. Moreover, a multiplexer was utilized to design basic PIM cells.

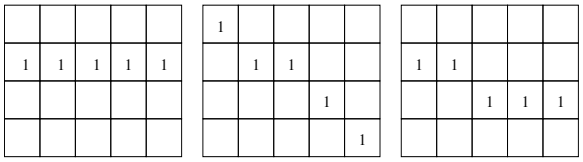


Figure 3.11: Akers 1-path